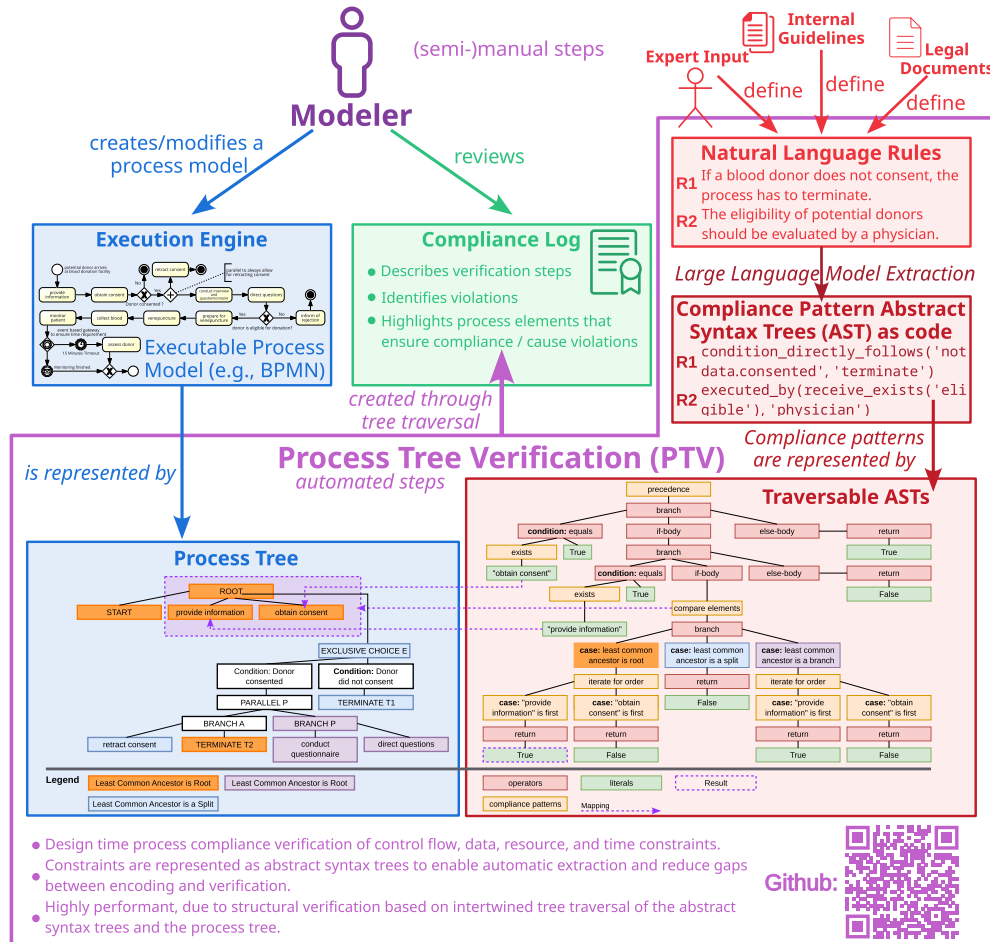


Graphical Abstract

Taming the Complexity of Design Time Process Compliance Through Tree-Based Multi-Perspective Verification

Johannes Loebbecke, Juergen Mangler, Stefanie Rinderle-Ma



Highlights

Taming the Complexity of Design Time Process Compliance Through Tree-Based Multi-Perspective Verification

Johannes Loebbecke, Juergen Mangler, Stefanie Rinderle-Ma

- Design time compliance verification of executable process models
- Polynomial time complexity due to structural verification of process trees
- Data, resources, and time constraints in addition to control flow
- During process modeling provide immediately traceable compliance results

Taming the Complexity of Design Time Process Compliance Through Tree-Based Multi-Perspective Verification

Johannes Loebbecke^{a,*}, Juergen Mangler^a, Stefanie Rinderle-Ma^a

^a*Technical University of Munich, TUM School of Computation, Information and Technology, Boltzmannstraße 3 Garching, 85748, Germany*

Abstract

Process compliance ensures that processes adhere to requirements imposed on them. Failure to comply with legal and industry regulations can result in substantial fines, legal consequences, and reputational damage. Compliance-by-design approaches thus aim to ensure that every process execution is compliant, which is essential in domains where violations are unacceptable. However, existing approaches fail to achieve the dream of compliance-by-design due to abstraction and implementation gaps between regulatory documents, designed process models, and executed processes as well as performance limitations of model checking. In particular, data, time, and resource requirements are often annotated to process models, but not enforced during execution. This work aims to achieve more substantial compliance-by-design assurances at an execution-level abstraction using structural verification of executable process trees (process tree verification, PTV). We provide formal results for the comparison of structural and behavioral verification regarding computational complexity and trustworthiness of verification results, which are complemented by a comparative evaluation of PTV with two model-checking approaches. PTV is prototypically implemented, applied to a data set, and evaluated regarding its usability through a qualitative survey.

Keywords:

*Corresponding author

Email addresses: `johannes.loebbecke@tum.de` (Johannes Loebbecke), `juergen.mangler@tum.de` (Juergen Mangler), `stefanie.rinderle-ma@tum.de` (Stefanie Rinderle-Ma)

1. Introduction

Process compliance is crucial for companies and organizations across all application domains [1]. Especially for highly-regulated application domains such as finance and healthcare, *compliance-by-design* [2], i.e., the modeling of compliance requirements and integration into the process model and connected enterprise applications, has proven valuable to avoid compliance violations leading to fines or other severe consequences. However, the *dream of compliance-by-design* [3] has yet to be realized due to the abstraction, terminology, and implementation gaps between legal documents, conceptual process models, and executable processes [4, 5, 6]. Moreover, the rapid change and evolving nature of legal and regulatory frameworks, e.g., in the financial domain, regulations change every 12 minutes¹, poses rarely addressed challenges on digital compliance management frameworks [3, 7].

All this requires a compliance verification approach that (i) bridges the gap between compliance requirements stated in regulatory documents and executable processes. It also requires (ii) performant and trustworthy verification techniques for execution-level data flow where data objects are provided by external services during runtime. Eventually, (iii) it must become possible to revisit process models that have been compliant-by-design if regulatory documents and, subsequently, the compliance requirements or the process model change. Then, an assessment of the state of compliance is crucial, particularly involving stakeholders such as compliance analysts. This work aims at addressing gaps (i)–(iii) through the following questions:

RQ1: How to represent compliance requirements extracted from text such that they can be verified over executable process models? Consider the running example process from the healthcare domain with associated requirements *R1* to *R4* as depicted in Fig. 1. The requirements are based on standards found in the blood guide² published by the European Directorate for the Quality of Medicines & HealthCare, which presents

¹<https://thefinanser.com/2017/01/bank-regulations-change-every-12-minutes>, accessed 2025-05-01

²<https://www.edqm.eu/en/blood-guide>, accessed 2025-02-19

standards for the preparation, use, and quality control of blood components in Europe. Figure 1 illustrates the representation and abstraction gaps: the process model is represented in BPMN³ and requirements R1–R4 are represented in text. Moreover, R4 is formulated at a different abstraction level than the tasks in the process model, imposing a constraint on the process without tying it to specific activity labels. Accordingly, the compliance representation has to consider the data, resource, and time perspectives in addition to the control flow perspective. Furthermore, the representation has to be both human and machine-readable to enable validation by domain experts and automatic extraction/verification. Finally, there can not be an interpretation gap between the representation and the verification methods that could lead to verifying a different property than intended.

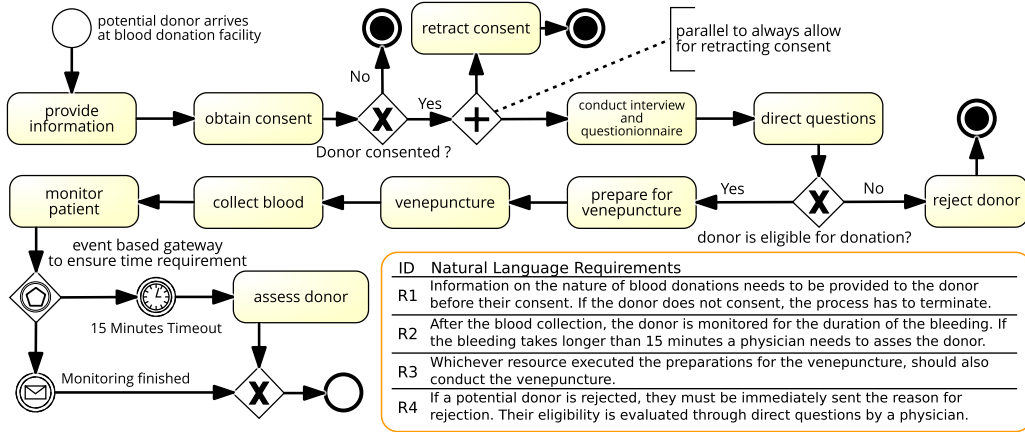


Figure 1: Example process (modeled in BPMN) and associated compliance requirements

RQ2 How to structurally verify compliance requirements over executable process models, and how to refine these process models? Verifying execution-level data flow of complex processes containing loops, parallel splits, and conditions that compare different data objects with a behavioral approach requires dealing with infinite sets of traces and the state explosion problem [8]. While abstraction techniques can reduce this problem [9, 10], the computational complexity still makes behavioral approaches unsuitable for live verification where requirements and process models regularly change, while the domains of data objects are infinite or unknown. Further-

³<https://www.bpmn.org>, accessed 2025-05-30

more, behavioral verification cannot directly identify the model elements that result in non-compliant traces. A structural approach checks the semantics of model elements (branch conditions, parallel splits, ...) to provide strong assurance of compliance without evaluating the entire state space. Structural approaches are highly performant [9, 11, 12] and offer traceable verification by highlighting which model elements violate/ensure the satisfaction of a requirement. This enables real-time feedback during modeling, which helps process modelers understand why compliance requirements are satisfied or violated, and supports them in creating compliant models, even for complex processes.

RQ3 How does structural verification compare to behavioral verification regarding computational complexity and trustworthiness of results? While structural verification does not require exploring the state space of a process model, it does require traversing the process tree to verify the constraint. Accordingly, it is essential to compare the computational complexity of the tree traversal required for structural verification with the complexity of state space exploration. Furthermore, state space exploration directly proves that a set of defined violating states cannot be reached, while structural verification identifies patterns in the process tree to ensure that a set of defined violating states cannot be reached. Accordingly, for structural verification results to be trustworthy, they require per-pattern proofs that ensure that the occurrence of a pattern in a tree prevents the violating states associated with this pattern from being reached during execution.

For addressing **RQ1**, we propose a formalization of compliance requirements covering control flow, time, data, and resources based on Abstract Syntax Trees (AST), which are a typical output format of Large Language Models (LLM) and can be directly verified without introducing an interpretation gap between structural model verification methods and requirements formalization. Compliance verification is based on an intertwined tree traversal of the (combined) AST reflecting compliance requirements of interest and the process tree reflecting the process model (**RQ2**). To answer **RQ3**, we formally analyze the computational complexity of this Process Tree Verification (PTV) approach and compare it with the computational complexity of recent model-checking-based approaches [10, 13]. While we do not provide proofs for all presented PTV patterns, we show a path towards proving correctness of PTV results through example proofs of important patterns. For the evaluation, the PTV approach is prototypically implemented and

applied to a composite data set of 11 process models with 46 requirements from different domains. Moreover, a user study with 13 participants shows that the approach can support process modelers in identifying compliance violations and refining the process model to restore compliance. Finally, the results of the formal computational complexity analysis are corroborated through a comparative evaluation with the same two model-checking-based approaches [10, 13].

This work extends our previous work in [14] in different ways. First of all, we provide formal proofs on computational complexity and result trustworthiness for structural compliance verification in PTV, addressing RQ3. We also add compliance patterns to improve expressivity of PTV and provide a comparison of PTV with two existing model checking approaches. Finally, this work features an extended analysis of limitations and threats to validity.

The paper is structured as follows: Section 2 discusses the requirements encoding while Sect. 3 contains the algorithms for AST traversal and process tree verification. Section 4 discusses computational complexity and proving correctness of structural verification. Section 5 presents the evaluation and Sect. 6 discusses related work. Finally, Sect. 7 discusses limitations and threats to validity while Sect. 8 summarizes the results of the Process Tree Verification (PTV) approach.

2. Compliance Requirements Encoding

As motivated in the introduction, existing approaches relying on specific logic notations, e.g., CTL [4], LTL [15], MDL [16], suffer from an abstraction gap between text and formalized requirements as well as between formalized requirements and algorithmic verification techniques. Moreover, these approaches are not unified, requiring modelers to learn different logic notations, which is undesirable, especially if they contain new, custom elements [17]. Finally, existing approaches lack focus on execution-level data flow [3], i.e., data exchanged between an engine and the implementation of a task invoked through an endpoint. Since we present a structural verification approach, we propose using abstract syntax trees (ASTs) to formalize compliance requirements (cf. Def. 2.1).

ASTs are widely used to represent programs or code snippets and benefit from extensive tool support. Furthermore, advances in AI technologies enable the automatic generation of complex ASTs from simple descriptions, which is

Requirements as ASTs (code form)

```

R1 precedence('obtain consent','provide information') and
   condition_directly_follows('not data.consented','terminate')
R2 timed_alternative('monitor patient','assess donor',900) and
   executed_by('assess donor','physician')
R3 executed_by_return('prepare for venepuncture') ==
   executed_by_return('venepuncture')
R4 condition_directly_follows('not data.eligible',send_exists('data.rejection')) and
   executed_by(receive_exist('eligible'),'physician')

```

Figure 2: Constraints of the Running Example as ASTs in code form

promising for automatic requirements extraction. While automatic extraction is not the focus of the paper, we conducted some initial experiments with the GPT-4 model and an adapted, existing prompting technique [18] to transform the textual requirements of the example process (cf. Fig. 1) into code form ASTs (cf. Fig. 2, both from a complete textual process description and natural language requirements⁴. The extracted requirements are preliminary w.r.t. consistency and correctness but always contain required activities, resources, time descriptions, data objects (α), and compliance patterns (CP)(θ).

AST formalization offers three further advantages, i.e., (a) AST tree traversal directly prevents injection attacks, where the requirements contain undesired code pieces since allowed functions and attribute accesses are restricted; (b) it reduces development effort while improving reliability and trustworthiness since sophisticated parsing and performant AST traversal are already developed for compilers and (c) compilers also give extensive feedback in case of syntax errors in the requirements which supports rule modelers. Furthermore, if a temporal logic for requirements representation is still desired (e.g., for formal proofs), a custom parser can be created to translate requirements between a logic system and an AST or vice versa.

Definition 2.1 (AST-based Compliance Requirements Encoding). With $L = \{\text{literals}(\alpha), \text{variables}(\beta), \text{operators}(\oplus), \text{function calls}(\theta)\}$ an AST is a finite, rooted, ordered tree $T = (V, E, \lambda)$ where:

- V is a finite set of nodes
- $E \subseteq V \times V$ is the set of ordered edges connecting the nodes
- $\lambda : V \rightarrow L$ is a labeling function that assigns each node a label

⁴Prompts/Results: <https://github.com/JohannesLbck/ProcessTreeVerify>

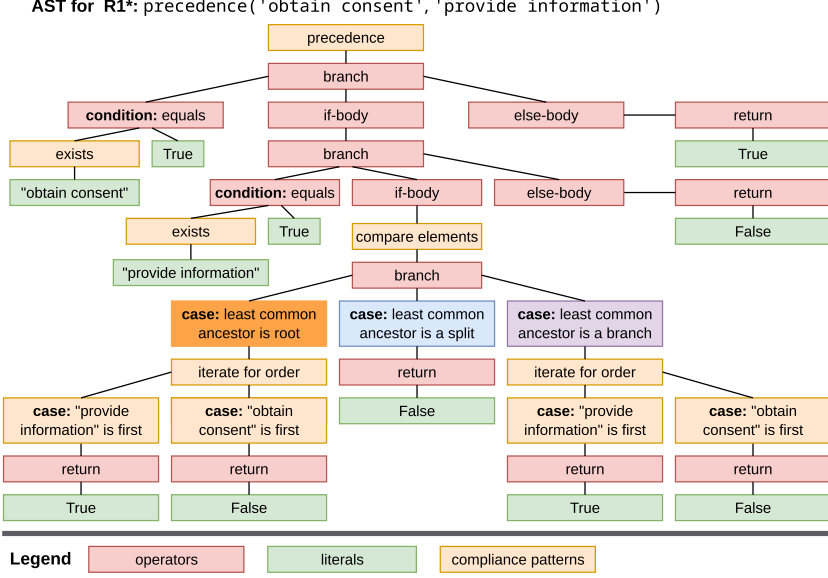


Figure 3: Precedence AST as complete Tree

To formalize requirements using an AST, literals contain all activity, resource, data object, and time descriptors, while variables can be freely assigned, operators are all standard operators (if, and, or, +, ==, ...), and function calls θ can correspond to any compliance pattern (CP), e.g., existence of activities. Figure 2 depicts the ASTs of $R1$ to $R4$ of the running example in code form and Fig. 3 the AST for the *precedence* CP required for $R1$. For simplicity, the *precedence* AST and other CP ASTs in this paper do not contain checks for correct data types and passing identical elements that are included in the implementation.

The literals also define the type of a requirement. Assume an AST T with literals a : if a contains only activity names, then T represents a pure control flow requirement; if a contains activity names and resource descriptors or only resource descriptors, then T represents a resource requirement; if a contains activity names and time descriptors or only time descriptors, then T represents a time requirement; if a contains data objects descriptors, then T represents a data flow requirement. Since the formalization is directly mapped to the implementation, there is no gap between formalized requirements and verification methods. Furthermore, the expressivity is not limited by the formal language but by the implemented verification methods, which can be freely combined using standard operators to model requirements. In

particular, the deontic modalities of obligations, permissions, and prohibitions can be modeled similarly to [19], by defining the prohibition of an AST A , as the obligation of $\neg A$, while if A is a requirement (it is obligatory), then $\neg A$ is prohibited and any AST that is not prohibited is permitted.

Based on CPs defined in literature, e.g., [20], we define two sets of CPs, i.e., Atomic Compliance Patterns (ACP) in Tab. 1 and extended compliance patterns (ECP) in Tab. 2. ACP are basic patterns that can be used in combination with literals in the AST, other ACP, and operators to represent any verifiable requirement, which we show by using the ACP to represent the ECP and CP found in related work (See Sect. 5). ACP `C`, for example, compares the positions of activities `a` and `b` in `tree`, representing the process (cf. Sect. 3). The `tree` argument is used to define the scope of a CP, such that a CP can be required in an entire tree or just a branch. If the `tree` argument is omitted, we assume that the CP is obligated for the entire process tree.

Table 1: Atomic Compliance Patterns (ACP). *Tree*, *a*, and *b* can all be any node

Atomic Compliance Pattern	Short	Returns	Description
Existence(<i>tree</i> , <i>a</i>)	<i>E</i>	Element/None	Node matching to <i>a</i> in <i>tree</i>
Compare(<i>tree</i> , <i>a</i> , <i>b</i>)	<i>C</i>	<i>P</i> , <i>X</i> , $\rightarrow$, $\leftarrow$	Checks relationship of nodes <i>a</i> and <i>b</i> in <i>tree</i>
Can_directly_follow(<i>tree</i> , <i>a</i> , <i>b</i>)	$\longrightarrow$	True/False	Can directly follow in some traces
Must_directly_follow(<i>tree</i> , <i>a</i> , <i>b</i>)	$\Longrightarrow$	True/False	Must directly follow in every trace
cancel_first(<i>tree</i> , <i>a</i> , <i>b</i>)	<i>CF</i>	Subtree/None	identified a classic event based gateway
cancel_last(<i>tree</i> , <i>a</i> , <i>b</i>)	<i>CL</i>	Subtree/None	identifies a cancel last gateway (cf. 3.2)
executed_by(<i>tree</i> , <i>a</i>)	<i>R</i>	[Resources]	Returns list of resources that execute an activity
timeouts_exists(<i>tree</i>)	<i>T</i>	[Timeouts]. <i>t</i>	List of timeout elements with the times as <i>t</i>
due_date_exists(<i>tree</i>)	<i>D</i>	[Due Dates]. <i>t</i>	List of due date elements with times as <i>t</i>
data_objects(<i>tree</i>)	<i>DO</i>	[Data Objects].(<i>s</i> , <i>r</i>)	Data objects, <i>s</i> for sends and <i>r</i> for receives
activity_data_checks(<i>tree</i> , <i>a</i>)	<i>DA</i>	[Data Objects].(<i>s</i> , <i>r</i>)	Data objects send(<i>s</i>)/received(<i>r</i>) by a activity
activity_data_rescue(<i>tree</i> , <i>a</i>)	<i>DR</i>	[Data Objects]	Data objects rescued when an activity fails
condition_finder(<i>tree</i> , <i>condition</i>)	<i>DC</i>	[Branches]	List of branches with matching condition
loops(<i>tree</i>)	<i>L</i>	[Loops]	List of Loop elements

The formal base of the ECP is ensured by representing each ECP using the ACP and First Order Logic (FOL). The proposed ACP do not constitute a minimal set, and it would be possible to reduce the set of ACP further by, e.g., replacing the existence patterns with a single $\exists$ at the cost of understandability. In general, since it is challenging to formalize requirements as ASTs using just the ACP, the ECP improve usability and automatic extraction.

AST-based encoding allows combining CPs such as `send/receive_exist` and `executed_by_return`, which return the process element sending/receiving a data object and the resource executing an activity, respectively. Verified independently, they are mapped to satisfied/violated using *truthy* evaluation, where `None` is evaluated as false and any object as true. However, since they return process elements, they can also be combined to model constraints at

Table 2: Extended Compliance Patterns (ECP) using FOL and ACP

Compliance Patterns for ASTs	Representation using FOL and ACP
exists(tree, a)	$E(tree, a)$
absence(tree, a)	$\neg E(tree, a)$
loop(tree, a)	$\exists x \in L(tree) : E(x, a)$
directly_follows(tree, a, b)	$\Rightarrow (a, b) \text{ or } \longrightarrow (a, b)$
leads_to(tree, a, b)	$\neg E(tree, a) \vee C(a, b) \equiv a \rightarrow b$
precedence(tree, a, b)	$\neg E(tree, a) \vee C(a, b) \equiv a \leftarrow b$
parallel(tree, a, b)	$C(a, b) \equiv a \parallel b$
exclusive(tree, a, b)	$C(a, b) \equiv a \text{ X } b$
executed_by(tree, a, r)	$\exists x \in R(a) : x \equiv r$
executed_by_return(tree, a)	$R(a)$
recurring(tree, a, time)	$\exists x \in L(tree) : E(x, a) \wedge \exists t \in T(x) : t.t \equiv time \wedge C(a, t) \equiv a \rightarrow t$
timed_alternative(tree, a, b, time)	$\exists x \in T(tree) : CF(x, a) \wedge x.t \equiv time \wedge E(CF(x, a), b)$
by_due_date(tree, a, time)	$\exists x \in D(tree) : x \equiv a \wedge x.t \equiv time$
max_time_between(tree, a, b, time)	$C(a, b) \equiv a \rightarrow b \wedge \exists x \in T(tree) : CL(x, b) \text{ and } x.t \equiv time$
activity_sends(tree, a, data)	$\exists x \in DA(tree, a) : x.s \equiv data$
activity_receives(tree, a, data)	$\exists x \in DA(tree, a) : x.r \equiv data$
send_exist(tree, data)	$\exists x \in DO(tree) : x.s \equiv data$
receive_exist(tree, data)	$\exists x \in DO(tree) : x.r \equiv data$
condition_directly_follows(tree, c, a)	$\exists x \in DC(tree, c) : E(x, a) \wedge \neg \exists y : E(x, y) \wedge C(a, y) \equiv a \leftarrow y$
condition_branch_eventually_follows(tree, c, a)	$\exists x \in DC(tree, c) : E(x, a)$
condition_eventually_follows(tree, c, a)	$\exists x \in DC(tree, c) : C(x, a) \equiv x \rightarrow a$
failure_eventually_follows(tree, a, b)	$\exists c \exists x \exists y (x \in DF(tree, a) \wedge x \in c \wedge y \in DC(tree, c) \wedge E(y, b))$
failure_directly_follows(tree, a, b)	as above $\wedge \#z : E(y, z) \wedge C(b, z) \equiv b \leftarrow z$

different abstraction levels. For example, *R3* compares executing resources, while *R4* (cf. Fig. 2) avoids tying the resource and data flow constraints to activity labels by combining the `condition_directly_follows`, `executed_by`, and `send/receive_exist` CP.

In general, condition patterns can be used to model requirements that enforce the existence of an activity in a subset of all traces defined by a condition *c* that compares one or more data objects to literals or other data objects. We differentiate by whether an activity should a) **directly follow** after *c* evaluates to true, b) **eventually follow only** when *c* evaluates to true, or c) **eventually follow** when *c* evaluates to true. The difference between b) and c) can be regarded as a scoping or *overcompliance* difference, where b) defines that the activity should *only* be executed in the traces defined by the condition, while c) allows the activity to be executed in more traces than defined. As used for *R4*, the activity can also be replaced by any CP that returns a model element.

While we include the `due_date`, `recurring` and `max_time_between` CP that can be identified in literature [21], we also propose the `timed_alternative` pattern, which can be found in operationalized requirements such as *R2* which is based on standard 3.6.2.6 in the Blood Guide. The `timed_alternative` CP subsumes the other time CP since it defines an alternative that has to be executed instead of an original activity if a time constraint is not held.

Finally, while we generally assume that values returned by endpoints are

according to the endpoint specifications, we do include two CP for handling potential errors in endpoints, i.e., `failure_eventually_follows(a, b)` and `failure_directly_follows(a, b)`. These CPs can be used to model requirements that enforce that an activity b should be (immediately) executed if activity a fails. They are required because the `condition_eventually/directly_follows` CP can only model constraints that require an activity to be executed in the event of a negative result returned by an endpoint and not following its outright failure. As an example, if b should be executed in case a potential donor is not eligible, then `condition_eventually_follows('not eligible', 'b')` is a correct encoding. If b should be executed in case the system determining the eligibility crashes, then `failure_eventually_follows('determine eligibility', 'b')` is the correct encoding.

3. AST-Based Process Tree Verification (PTV)

As discussed the formalization of compliance requirements as ASTs bears several advantages. In this section, we focus on how the AST representation can be leveraged for compliance verification of executable processes. To this end, we opt for an executable representation of process models as *process trees* to enable the structural verification of tree-based requirements on a tree-based model.

A process tree is an ordered, tree-based data structure inspired by the Refined Process Structure Tree (RPST) [11], where each node can be either a branch or a leaf. Figure 4 depicts part of the process tree of the running example. Similar to [22], we formally define process trees recursively.

Definition 3.1 (Process Tree). Let Σ be the set of activities with $\tau \notin \Sigma$ denoting the silent activity.

- a with $a \in \Sigma \cup \{\tau\}$ is a process tree M
- Let $M_1, \dots, M_n$ with $n > 0$ be process trees, with $\rightarrow$ denoting a sequence, $\times$ denoting an choice, and $\wedge$ denoting a parallel, then $\otimes \in \{\rightarrow, \times, \wedge\}$ $\otimes(M_1, \dots, M_n)$ is a process tree
- Let M be a process trees and let $\circlearrowleft$ denote a loop relationship, then $\circlearrowleft(M)$ is a process tree

To enable process execution, we additionally differentiate between $\rightarrow$, $\rightarrow_p$ and $\rightarrow_e$ to explicitly define parallel and exclusive branches as well as $\wedge$, $\wedge_f$,

and $\wedge_l$ which differentiate between a pure parallel ($\wedge$), a standard event based gateway ($\wedge_f$) where the first activity that finishes cancels the other branches, and a last cancel ($\wedge_l$) where the first branch that finishes cancels the other branches. Their semantics are included the following language definition.

Definition 3.2. Let $\sum^*$ denote the set of traces composed of activities in $\sum$, with the empty trace represented as $\langle \rangle \in \sum^*$. Length of a trace σ is denoted by $|\sigma|$, its k -th activity is denoted by $\sigma[k]$. $\sigma[k \dots]$ refers to the partial trace σ after (excluding) the k -th activity and $\sigma[\dots k]$ refers to the partial trace σ up to the (excluding) k -th activity. We use the Kleene star $*$ operator and, for any two traces σ_1, σ_2 , we denote the following operations: concatenation $\sigma_1 \cdot \sigma_2$ and interleaving $\sigma_1 \diamond \sigma_2$, which can both be extended to trace sets as well. We define the language L of a process tree M recursively as:

- (leaf node:) let $M = a$ with $a \in \sum$ then $L(M) = \{\langle a \rangle\}$
- (silent transition:) let $M = \tau$ then $L(M) = \{\langle \rangle\}$
- (sequence:) let $\Rightarrow \in \{\rightarrow, \rightarrow_p, \rightarrow_e\}$ and $M \Rightarrow (M_1, \dots, M_n)$ then $L(M) = \bigodot_{i=1}^n L(M_i)$
- (choice:) let $M = \times(M_1, \dots, M_n)$ then $L(M) = \bigcup_{i=1}^n L(M_i)$
- (parallel:) let $M = \wedge(M_1, \dots, M_n)$ then $L(M) = \diamond_{i=1}^n L(M_i)$
- (loop:) let $M = \circ(M_1)$ then $L(M) = M_1^*$
- (event-based-gateway:) let $M = \wedge_f(M_1, \dots, M_n)$ then $L(M) = \{\sigma \in \{\sigma_k\} \diamond \{\sigma_i[1] \mid \sigma_i \in \bigcup_{i \in \{1, \dots, n\} \setminus \{k\}} L(M_i) \mid \exists k \in \mathbb{N}_0, \sigma_k \in L(M_k)\}\}$
- (last cancel:) let $M = \wedge_l(M_1, \dots, M_n)$ then $L(M) = \{\sigma \in \{\sigma_k\} \diamond \{\sigma_i[: -2] \mid \sigma_i \in \bigcup_{i \in \{1, \dots, n\} \setminus \{k\}} L(M_i) \mid \exists k \in \mathbb{N}_0, \sigma_k \in L(M_k)\}\}$

Accordingly, the process tree of the running example a shown in Fig. 4 can be formalized as $\rightarrow$ ("START", "provide information", "obtain consent", $\times(\rightarrow_e(\wedge(\rightarrow_p$ ("retract consent", "TERMINATE T2"), $\rightarrow_p$ ("conduct questionnaire", "direct question", ...))), $\rightarrow_e$ ("TERMINATE T1")), ...) where ... refers to the rest of the process tree required for the running example process in Fig.1 that is not visualized as a tree in Fig. 4.

Accordingly, leaf nodes represent tasks such as activities, terminates, and events like timeouts or stops. Branches represent control flow splits (exclusive choices, parallels, ...) and exclusive/parallel branches, which require a

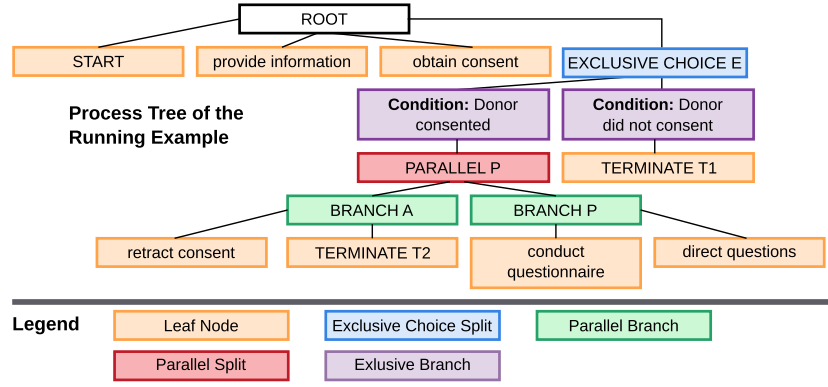


Figure 4: Snippet of the process tree of the running example

fitting split as a parent. Notably, tree edges do not represent the edges of the process model, as the execution order is implied by the tree order using depth first pre-order traversal rather than the edges, which describe a parent-child relationship between two nodes. Activities **provide information** and **obtain consent**, for example, are not connected by an edge since they are both leaf nodes. To enable process execution, a process tree contains implementation details (endpoint addresses, data objects reads/writes, ...), which are annotated to leaf nodes as nodes that do not affect the execution order. Key advantages of the process tree representation are (a) enforced syntactical correctness of the process model, (b) trivial insertion and deletion [23], and (c) performant transformation between workflow graph and process model [11].

3.1. General Verification and Control Flow Patterns

Assuming a set of ASTs representing different requirements and a process tree representing the executable process model, the basic idea of the verification approach is to iterate over the set of ASTs and traverse each AST until we find CPs in each AST that are verified by traversing the process tree. The verification instruction for a CP is encoded in the AST of this CP. Once a complete AST has been traversed, the return value is evaluated towards **satisfied** (true) or **violated** (false) while providing an assurance level and a log, that tracks the verification steps and highlights cases in which the verification results can be incorrect (cf. Sec. 3.2 and Sec. 4). Figure 5 presents the process tree from Fig. 4 in Fig. 5c) next to the precedence AST from Fig. 3 in Fig. 5b) to illustrate the intertwined tree traversal used for PTV.

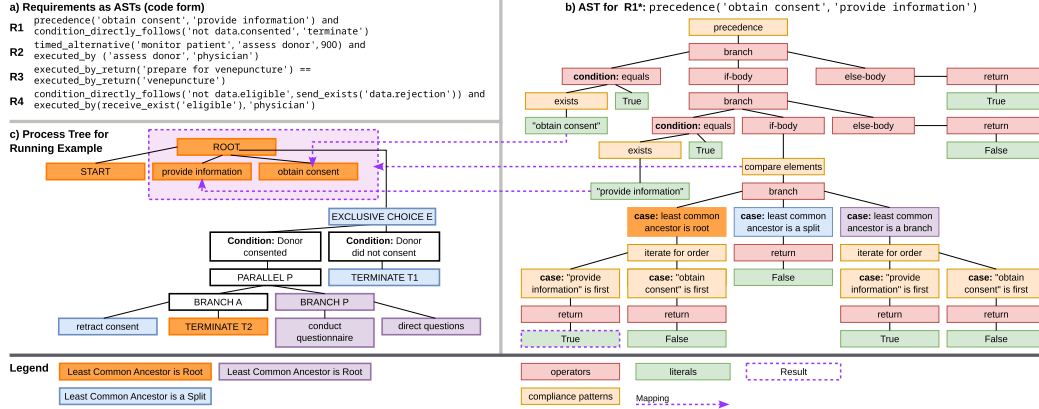


Figure 5: Requirements as ASTs containing CPs which map to the Process Tree

Algorithm 1 shows a generic AST traversal algorithm used for verifying a requirement formalized as an AST by matching the CP contained in the AST to a process tree. Take Fig. 5b) showing the AST of $R1^*$: `precedence('obtain consent', 'provide information')` to be verified over the process tree in Fig. 5c). Algorithm 1 traverses the AST and first finds the existence CP for task `obtain consent`. Traversing the process tree, `obtain consent` is matched. Continuing AST traversal, the next CP to be found is the existence of `provide information`, which is again matched in the process tree. Finally, AST traversal checks the precedence relation based on the compare elements CP in the AST by identifying the least common ancestor (LCA) [24] of `obtain consent` and `provide information` in the process tree.

Algorithm 1 General AST Traversal Algorithm

Require: Compliance Rule

Ensure: True/False

Tokenize (extract α , β , $\oplus$ and θ)

Build AST ($T = (V, E, \lambda)$ where $\lambda : V \rightarrow L = \{\alpha, \beta, \oplus, \theta\}$ and $(n, m) \in E$ is an edge $n \rightarrow m$)

procedure VISITNODE(Node)

 if $\lambda(\text{Node}) \in \alpha$ then return : $\lambda(\text{Node})$

 else if $\lambda(\text{Node}) \in \beta$ then return : value of $\lambda(\text{Node})$

 else if $\lambda(\text{Node}) \in \{\oplus, \theta\}$ then return $\lambda(\text{Node})(\{\text{VisitNode}(v) \mid v \in V, (Node, v) \in E\})$ ⁵

 end if

end procedure

⁵Compilers check which children need to be visited for which operator.

When traversing the tree upwards from two nodes, the LCA can be used to identify whether two nodes are in a shared parallel/exclusive branch (LCA is the branch, highlighted purple in Fig. 5), in different parallel/exclusive branches (LCA is a split, highlighted blue), or the nodes do not share any splits (LCA is root, highlighted yellow). The compare algorithm (Listing 1) uses the LCA property to first identify whether any two nodes/subtrees are on different branches of the same split (mutually exclusive or in parallel). Otherwise, a node appearing first in the ordered tree will always precede the other in every trace if both appear. Since the LCA of **obtain consent** and **provide information** is the root node, the tree order (in depth first, pre-order tree traversal order, shortened to tree order from now on) is used to confirm the precedence relationship.

Listing 1: Pseudo code algorithm for comparing tree elements; Top down approach.

```

1 def compare_ele(tree, node1, node2):
2     shared_ancestors = get_shared_ancestors(tree, node1, node2)
3     if shared_ancestors[-1].type() == x:
4         return "node1_and_node2_are_on_different_exclusive_branches"
5     elif shared_ancestors[-1].type() == ^:
6         return "node1_and_node2_are_on_different_parallel_branches"
7     ## Otherwise, the nodes are neither in parallel/mutually exclusive, so depth first
8     → tree order decides the control flow
9     for node in tree.traverse():
10        if node in {node1, node2}:
11            return node ## The node that appears first is returned

```

Figure 5b) shows the simplified AST for the **precedence** pattern since more complex CP require large ASTs, and the LCA concept is essential for several CP (see Tab. 2). The complete **precedence(M,a,b)** algorithm can be seen in Lst. 2. It additionally includes a *Most Common Ancestors* (MCA) check in the case that *b* is the child of a exclusive choice $\times$, in which case *a* and *b* have to be on the same branch to ensure compliance. The following subsections discuss the CP of the other perspectives.

Listing 2: Pseudo code algorithm for precedence pattern

```

1 def precedence(M, a, b):
2     if exists(M,a):
3         if exists(M,b):
4             compare = compare_ele(M,a,b) # LCA Check
5             ancestorsA, ancestorsB, shared = get_shared_ancestors(M,a,b)
6             if compare == b: # LCA check returns whichever appeared first or None
7                 if  $\times \in$  ancestorsB:
8                     if shared[0]  $\in \{\rightarrow, \rightarrow_p, \rightarrow_e\}$ : return True # MCA Check
9                     else: return False
10                else: return True
11            else: return False
12        else: return False
13    else: return True

```

3.2. Data Patterns

In typical process engines [23], data objects can be accessed (1) in preparation before they are sent to an endpoint, (2) when they are sent as argu-

ments towards an endpoint, (3) when results returned from an endpoint are written to data objects, and (4) when an endpoint returns an error message and (partial) results are rescued and written to data objects. Accordingly, for the **send_exist** CP, the verification has to search for the data objects in the (1) preparation and (2) argument fields, while for the **receive_exist** CP, it has to check the (3) return and (4) rescue fields in the process tree.

For the **condition_finder** CP, the condition c_r of a requirement should ideally match to a condition c_t of an exclusive branch in the process tree only if, for every potential data object state where c_r returns TRUE, c_t also returns TRUE. However, comparing arbitrary-condition behavior requires exploring an infinite state space when considering data objects with infinite domains, such as strings [10], or when c_t compares multiple data objects returned by endpoints whose data domains are unknown. Since both cases are common, we instead rely on a heuristic to identify perfectly matching conditions in most process trees, while informing the process modeler of potential errors using an assurance level with explanations. Accordingly, the **condition_finder** distinguishes three cases with different assurances:

- (1) $c_r \equiv c_t$: In case of a perfect match, i.e. the branch conditions equals the requirement condition, the branch is returned with full assurance
- (2) $c_r \in c_t$: If c_r is part of c_t , the branch is returned with reduced assurance since the complete condition might have different behavior.
- (3) $c_r \in \neg c_t$: If c_r is part of the negation of c_t , the default branch of the same split is returned with significantly reduced assurance, since, assuming a split with branch conditions $(c_1, c_2, \dots, c_n)$ then the condition of the default branch is the negation of the disjunction of all c_n , instead of a single $\neg c_n$.

If a c_r is not mapped using any of the cases for all branches in the process, it returns None. Since the condition patterns check the existence of required activities on all branches matched using the **condition_finder** in the order of the cases above, the verification will return results with as high assurance as possible while highlighting which conditions have to be re-checked, as they were matched using cases (2) and (3).

Condition CP then uses the information about data object reads and the **condition_finder** to ensure compliance properties that depend on data object states. In particular, data object reads are checked in a **condition_impacts** pattern to ensure that an identified branch exists after the data

object states can be reached. Consider the `condition_eventually_follows(M,c,a)` CP as implemented in Listing 3: As the satisfaction of its condition c depends on data objects $D \in c$, c can be satisfied after every activity α that receives any $d \in D$ (either in the (3) return or the (4) rescue processing fields) after α is executed. Accordingly, its algorithm in Lst. 3 uses the condition finder pattern to identify a fitting branch B and, if the required activity a is found on B , it checks, for each α identified using `condition_impacts`, whether it is prior to the branch. In the case that no activity receives any d , it is assumed that the data objects are set from the Start activity (e.g., as M is a sub process), so the eventually follows property is satisfied for all positions of B . The `branch` scope is set as the default, as it maps to the **eventually follows only** relationship discussed in Sec. 2. The `global` scope maps to the classical **eventually follows** relationship, i.e., it accepts *maximally overcompliant* [25] processes since the CP returns True, even if a is executed in every trace, regardless of whether c is ever satisfied or not (cf. vacuous satisfaction [26]). The `condition_directly_follows(M,c,a)` CP has to additionally consider that a has to directly follow after every α which is achieved through a `multi_condition_finder(M,c)` that identifies multiple fitting branches $\mathbb{B}$, where each $B \in \mathbb{B}$ has to follow after each α .

Listing 3: Pseudo code algorithm for condition eventually follows pattern

```

1 def condition_eventually_follows(M, c, a, scope = 'branch'):
2     B = condition_finder(M, c)
3     if scope == 'branch':
4         if B:
5             if exists(B,a):
6                 A = condition_impacts(M, c) # Identifies all potential activities that
3                 can satisfy c
7                 for  $\alpha$  in A:
8                     if not leads_to(M,  $\alpha$ , B): return False
9                 return True
10            else: return False
11        else: return False
12    elif scope == 'global':
13        ## handled in the same manner but 'a' can be anywhere after every  $\alpha$ 

```

The send/receive and condition patterns can also be combined to create failure-handling patterns to verify the existence of alternatives that are executed in case an activity fails. As with the standard condition patterns, we differentiate between `failure_eventually_follows(a,b)` and `failure_directly_follows(a,b)`. The main difference to the standard condition patterns is that for the failure handling patterns, the condition is not given in the requirement, but rather the data objects that are rescued in the (4) rescue processing field of a are used as potential conditions c for the `condition_finder(c)`. Accordingly, the failure CP return true if b eventually/directly follows in a branch that checks eventually/directly after a whether a

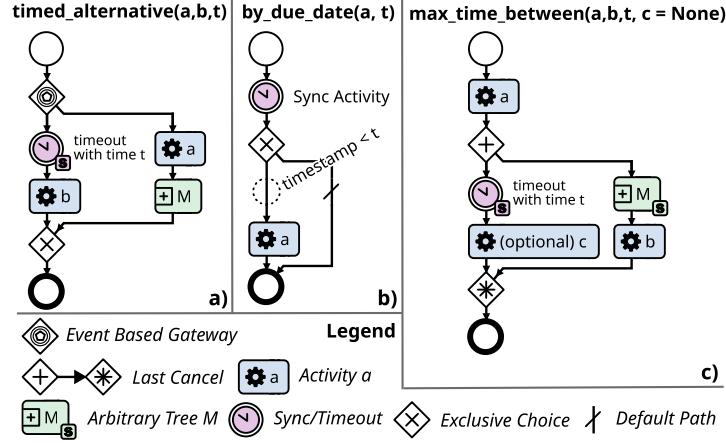


Figure 6: Explicitly enforcing different time patterns

data object rescued from `a` returns true.

3.3. Time Patterns

Enforcing time constraints at design time is challenging, as process trees are used to orchestrate distributed systems without globally trusted clocks and due dates can be enforced both in the endpoint and the process model. Accordingly, many design time compliance approaches rely on semantic annotations [2, 16], where time properties are annotated to activities without considering how they are enforced at run time. While we support semantic annotations, we further provide patterns that verify whether a process model explicitly enforces time-related CPs by comparing the positions and data reads/writes of sync and timeout endpoints. We illustrate explicit enforcement strategies for the three temporal patterns in Fig. 6. It contains process fragments that ensure the temporal constraint is held when they appear in the process, with example activities `a` and `b`, a time(stamp) `t`, and a sub process `M` that represents arbitrary process trees. In addition, start and end events can also be seen as arbitrary process trees. As an example, the `timed_alternative` is enforced by identifying the fragment in Fig. 6a). It contains a event based gateway, followed by `a` in one branch and a timeout in the other branch. If the timeout finished before `a` is finished, then `b` has to be executed, while the branch with `a` is canceled.

Accordingly, the algorithm for verifying the `timed_alternative` CP (Lst. 4) first ensures that both `a` and the alternative `b` exist, since absence of either

violates the requirement. If both are present, the CP checks all timeouts in the process tree to find those in a event based gateway (or cancel first) relationship with **a**. For each, it checks if the timeout’s parameter matches the required time, and if so, verifies if **b** appears on the alternative branch after the timeout. The **max_time_between** CP in Fig. 6c) instead identifies a timeout in a *cancel last* relationship with **b**. Accordingly, if the time between *a* and *b* exceeds *t*, the timeout cancels the other branch, so *b* can never be started too late. The **due_date** CP in Fig. 6b) checks for a branch condition containing the data object returned by a sync activity before the time-constrained activity. Finally, the **recurring(a, t)** CP (not illustrated) searches for a loop structure which contains a subtree/activity *a* and a timeout with value *t* after *a*, such that *a* is executed every *t* seconds.

Listing 4: Simplified pseudo code algorithm for the timed alternative pattern

```

1 def timed_alternative(tree, a, b, time):
2   if not exists(tree, a) then return: false
3   if not exists(tree, b) then return: false
4   for timeout in timeouts_exists(tree):
5     parallel = cancel_first(tree, timeout, a)
6     if parallel is not None:
7       if timeout.t == time:
8         return: exists(parallel, b)
9   return: false ## No timeout was found to enforce the timed alternative

```

3.4. Resource patterns

The challenge with resource CP is that the process tree does not and should not enforce how resources are assigned to endpoints. For example, requirement *R4* mandates that a physician assesses donor eligibility through direct questions. In an executable process, this could be ensured through (1) an endpoint that requires a resource assignment in the REST call header/body, (2) an endpoint implemented to only be executable by a physician, or (3) a resource organization platform such as [27]. For (1), the resource has to be checked in the REST call header/body, while for (2), the resource is assigned in the endpoint implementation, so the requirement cannot be verified in the executable process. Finally, for (3), the compliance rule would be integrated into the allocation algorithms. Restricting the way a process tree assigns resources to endpoints would reduce the flexibility of process tree based execution. For a company-specific compliance solution, we can add such a restriction, reducing flexibility but enabling highly reliable verification. For a generic solution, we can only verify resource CPs using semantic annotations, similar to existing approaches [17], without assurance that the annotations are enforced at runtime. For this annotation solution, we do not

differentiate between resources and related concepts (e.g., roles, departments, teams, ...) since they can be annotated analogously.

4. Computational Complexity and Correctness

This section focuses on the computational complexity as the central advantage and the challenge in proving correctness of verification results as the central disadvantage of structural verification. We define that structural verification results are *correct* if, given a process tree M , the language of the process tree $L(M)$ contains no violating traces w.r.t. to the behavioral specification of a CP P *iff* (logical equivalence) or *if* (implication) structural verification of that pattern P returns true. We distinguish *iff* and *if* assurance: If and only if structural verification returns TRUE (*iff* assurance), the tested process cannot generate violating traces w.r.t the specification of the tested pattern. If structural verifications returns TRUE (*if* assurance), the tested process cannot generate violating traces, but it does not ensure that if structural verification returns FALSE, the tested process can generate violating traces.

The correctness of verification results for model-checking-based verification (behavioral) is given by state exploration which is a NP-complete problem [28]. Correctness of structural verification requires formal proofs for each CP; selected example correctness proofs are presented in Sec. 4.2. The computational complexity analysis for PTV presented in Sec. 4.1 shows that structural verification has linear to polynomial time complexity depending on the implementation. Table 3 contains an overview of key ACP and ECP with their computational complexity class, their assurance type and a behavioral definition of violating cases. The complexity classes are based on the number of elements in the tree n , while assuming that the number of resources n_{re} and the number of data objects n_d is comparable but less than n , such that $O(n + n_d) = O(n)$ and $O(n \times n_{re}) = O(n^2)$. The final computational complexity also considers the number of compliance requirements r , leading to a final complexity of $O(r * n^2)$.

In Tab. 3, we use the following notation to behaviorally define what constitutes a violation for each pattern: We define σ_a as an arbitrary trace which

⁶Complexity assumes that there are less data objects than tree elements

⁷Complexity assumes that there are nearly as many, but not more resources than tree elements

Pattern	$O()$	A	Violation if
<code>exists(M,a)</code>	n	iff	$\nexists \sigma_a \in L(M)$
<code>absence(M,a)</code>	n	iff	$\exists \sigma_a \in L(M)$
<code>activity_sends(M,a,d)</code>	n	iff	$\exists \sigma_a \in L(M) : a_d \notin \sigma_a$
<code>activity_receives(M,a,d)</code>	n	iff	$\exists \sigma_a \in L(M) : a^d \notin \sigma_a$
<code>send_exist(M,d)</code> ⁶	n	iff	$\exists \sigma_a \in L(M) : \nexists a \in \sigma_a : a_d$
<code>receive_exist(M,d)</code> ⁶	n	iff	$\exists \sigma_a \in L(M) : \nexists a \in \sigma_a : a^d$
<code>leads_to(M,a,b)</code>	n	if	$\exists \sigma_a \cdot \sigma_{\neg b} \in L(M)$
<code>precedence(M,a,b)</code>	n	if	$\exists \sigma_{\neg b} \cdot \sigma_a \in L(M)$
<code>must_directly_follow(M,a,b)</code>	n^2	if	$\exists \sigma_{\neg a} \cdot a \cdot \sigma \in L(M) : \sigma[0] \neq b$
<code>executed_by_identify(M,res)</code> ⁷	n^2	iff	$\forall a \in \Sigma, \nexists \sigma_a \in L(M) : a.res = res$
<code>executed_by(M,a,res)</code>	n	iff	$\nexists \sigma_a \in L(M) : a.res = res$
<code>executed_by_return(M,a)</code>	n	iff	$\nexists res \in R, \exists \sigma_a \in L(M) : a.res$
<code>timed_alternative(M,a,b,t)</code>	n	if	$\exists \sigma_a \in L(M) : a(t) > t$
<code>by_due_date(M,a,t)</code>	n	if	$\exists \sigma_a \in L(M) : a(t) > t$
<code>max_time_between(M,a,b,t)</code>	n	if	$\exists \sigma_a \cdot \sigma_b \in L(M) : b(t) - a(t) > t$
<code>condition_directly_follows(M,c,a)</code>	n	if	$\exists \sigma \in L(M) : \exists i \in \mathbb{N}_0 : sat(\sigma[i], c)$ $\wedge \sigma[i+1] \neq a$
<code>condition_eventually_follows(M,c,a)</code>	n	if	$\exists \sigma \in L(M), \exists i \in \mathbb{N}_0, \forall l \in \mathbb{N}_0 : sat(\sigma[i], c)$ $\wedge \sigma[i+l] \neq a$
<code>condition_leads_to_absence(M,c,a)</code>	n	if	$\exists \sigma \in L(M), \exists i, l \in \mathbb{N}_0 : sat(\sigma[i], c)$ $\wedge \sigma[i+l] = a$

Table 3: Important CP analyzed regarding their worst case computational complexity ($O()$) and assurance level (A), Additional patterns on GitHub

contains activity a at any point and $\sigma_{\neg a}$ as an arbitrary trace which does not contain a . a^d denotes that activity a receives data object d and a_d that a sends data object d . We further use $a.res$ to express that activity a is executed by resource $res \in R$ with R referring to the set of resources. We still use indexed accesses to traces such that $\sigma[0]$ refers to the first element in a trace σ . For conditional patterns, we use a satisfaction function $sat(a, c)$ to define that after activity a receives (finalizes or rescues) data objects, condition c is satisfied. Finally, $a(t)$ refers to the timestamp at which point a is executed in any σ_a . For example, assuming a process M and an activity a , the `exists(M,a)` pattern is incorrect if it returns TRUE while $\nexists \sigma_a \in L(M)$. Furthermore, since the pattern fulfills *iff* assurance, it is also incorrect if the CP returns FALSE while $\exists \sigma_a \in L(M)$. It is important to note that we use an existence notion where a only has to appear in $1 \dots n$ traces and not all n traces for the CP to be satisfied. The $\forall$ existence notion can instead be enforced through `leads_to('Start', a)` as well as `precedence('End', a)`. The violation patterns are independent of the verification strategy and can also be used to proof correctness of other verification approaches.

4.1. Computational Complexity Analysis

We estimate the computational complexity of PTV on tree M with respect to the number of tree elements n and the number of requirements r . As the basic verification strategy is to iterate over all r and then execute the corresponding constraints formalized as ASTs, the computational complexity is bounded by r multiplied by the upper bound of the computational complexity of any constraint, denoted by O_c . Every AST constraint can be composed of arbitrary many patterns, but from a complexity perspective each composition equals a sequential execution of an additional pattern (cf. Alg. 1), so O_c is bounded by the largest O of any pattern.

By estimating the computational complexity of every pattern, we can identify that the majority of patterns are in $O(n)$ while some patterns are in $O(n^2)$ in specific worst cases. Accordingly, the complete worst case computational complexity of PTV is $O(r * n^2)$. Here it is important to note that the average computational complexity can be assumed to be in $O(r * n)$, as patterns are only in $O(n^2)$ with unrealistic worst case assumptions. For example the `directly_follows(a,b)` pattern is only in $O(n^2)$ if every other activity aside from `a` and `b` are terminate activities and the process does not contain exclusive choices or parallel branches. The only process where this applies contains just `a` and `b` in sequence, so $n = 2$. Detailed analysis for every pattern can be found on GitHub⁸.

Using structural verification it is theoretically feasible to process the constraint list into a single set of properties and check all properties in a single iteration over the process tree (i.e., $O(n)$), compared to iterating over the process tree for each constraint. However, this would require significantly more (theoretically infinite) memory with corresponding memory accesses and complicate the assurance logging which improves usability and prevents injection attacks (cf. Sec. 5).

As a comparison, in a journal extension to [9], [10] highlight an efficient state exploration approach supporting data objects with a worst case complexity of $O(e * r * 3^l * l * u^w)$, where e is the number of traces, r is the number of requirements, l is the maximal trace length, w is the number of write operations and u is the domain of these write operations. Comparing this to PTV complexity, we can identify that the number of traces in several cases

⁸https://github.com/JohannesLbck/ProcessTreeVerify/blob/master/Complexity_Estimation.md accessed 2026-01-21

turns out as $|L(M)| > n$, e.g., in case all activities are in parallel. Similarly, the maximal trace length in the “all parallel” case is the amount of elements such that $l \equiv n$. As for our own estimation, we assume that the number of data object writes is similar to, but not larger than n , $w \equiv n$ holds.

Accordingly, the computational worst case complexity of [10] is lower bounded by $O(r * n^2 * 3^n * u^n)$ compared to our worst case complexity of $O(r * n^2)$. Similarly, [29] who verify whether process models adhere to constraints encoded as goal models, highlight a complexity of $O(|Q| \times (|S_C| + m))$ in a preprint extension [13]. Here, $|Q|$ refers to the amount of qualities of the goal model which can be seen as comparable to r ; m is the number of transitions in a Labeled Transition System (LTS) combined from a goal model and a process model, and S_c is the set of states that the combined model can reach. Through the combination of the LTS, $S_C = S_p \times S_g$ holds where S_p is the amount of states of the process model and S_g is the amount of states of the goal model. While both S_g and S_p can be potentially infinite, we know that the number of states of the goal model will be comparable to the number of requirements so we assume that $S_g \equiv r$. Accordingly, the worst case complexity is $O(r * ((S_p * r) + m))$ or $O(r * m + r^2 * S_p)$. The comparative evaluation in Sec. 5.3 supports the results of this complexity analysis.

4.2. Proofing Correctness of Structural Verification

The first step in proving the correctness of the CPs is to distinguish patterns that fulfill *iff* assurance and patterns that fulfill *if* assurance. *Existence* patterns require a specific element to exist in the process tree, such as `exists(M, a)` which returns true if activity a exists in the tree M or `activity_sends(M, a, d)` which returns true if a exists in M and sends data object d . For these patterns *iff* correctness can be shown by analyzing how the identified elements constrain the language $L(M)$ of a process M .

Resource patterns rely on annotations to activities. Hence, verification results are only correct if assuming that the annotations hold at run time. With this assumption, resource patterns equal existence patterns and accordingly fulfill *iff* assurance.

Temporal patterns fulfill *if* assurance, as the CPs require patterns that enforce the temporal constraints in the model, but temporal constraints can also be enforced in the endpoint implementations. For example, `timed_alternative(M, a, b, t)` is enforced through a timeout followed by b in an event-based gateway with a as discussed in Sec. 3.3. However, it can also

be enforced by setting up the a endpoint to check the time and execute a different service corresponding to b if t has passed.

Similarly, even if we assume that the correct branch is identified (cf. 3.2), *Data Conditions* CP are only verified with *if* assurance, since we check for a fitting branch after each α after which c can be satisfied. As an example of when *iff* assurance is violated, consider a process M that never actually satisfies c , while it has an α after which c could be satisfied, and no branch B with a fitting condition c . In this case, the condition CPs would return False, as they do not identify a fitting branch, while the actual requirement is vacuously satisfied.

Branches that are never/always executed also lead to *if* correctness of control flow patterns such as **precedence**(M, a, b) which returns false in the case that b is on an exclusive branch B without a while B is always executed. Hence, $\forall \sigma \in L(M) : a \in \sigma \implies \sigma(a) > \sigma(b)$, i.e. b is executed before a in every trace where a appears. As the proof for each CP is directly connected to its algorithm, we present example proofs for the existence CP, the precedence CP (Lst. 2), and the condition eventually follows CP (Lst. 3) to illustrate how to proof correctness of structural verification.

Theorem 1. Let M be a process tree and let a and b be activities. Let σ be an arbitrary trace, σ_a and σ_b be arbitrary traces containing activities a and b respectively. Let further $\sigma_{\neg a}$ and $\sigma_{\neg b}$ be arbitrary traces not containing a and b respectively. Further the notations defined in Def. 3.2 and Sec. 4 are used such that $\sigma[\sigma(a)] = a$. Then the following statements hold:

Statement 1: **exists**(M, a) returns true $\Leftrightarrow \exists \sigma \in L(M) : a \in \sigma$

Statement 2: If **precedence**(M, a, b) returns true, it holds that $\forall \sigma \in L(M) : a \in \sigma \implies \sigma(a) > \sigma(b)$

Statement 3: If **max_time_between**(M, a, b, t) returns true, it holds that $\forall \sigma \in L(M) : a \wedge b \in \sigma \implies b(t) - a(t) < t$

Statement 4: Assuming that the **condition_finder** pattern identifies the correct branch, if **condition_eventually_follows**(M, c, a) returns true, it holds that $\forall \sigma \in L(M), \exists i \in \mathbb{N}_0 : sat(\sigma[i], c) \implies \sigma(i) < \sigma(a)$

Proof. We proof all statements by contradiction:

Statement 1: We first show $\Rightarrow$ (Statement 1.1) and then $\Leftarrow$ (Statement 1.2).

Statement 1.1: Assume M and a such that `exists(M,a)` returns true, but $\nexists \sigma \in L(M) : a \in \sigma$. `Exists(M,a)` returns true iff there exists a leaf node a in the process tree. Therefore, according to Def. 3.2, $\exists \sigma \in L(M) : a \in \sigma$ which contradicts with the assumption, so Statement 1 is true.

Statement 1.2: Assume M and a such that $\exists \sigma \in L(M) : a \in \sigma$, but `exists(M,a)` returns false. According to Def. 3.2 the only way for a to exist in a trace σ is if a is a leaf node in M . However, if a is a leaf node in M , then `exists(M,a)` returns true.

Statement 2.1: Assume M , a , b such that `precedence(M,a,b)` returns true, but $\exists \sigma_{-b} \cdot \sigma_a \in L(M)$. `Precedence(M,a,b)` returns true in three cases:

Case 1: `exists(M,a)` returns **False**, which, due to Statement 1.1 implies that $\nexists \sigma \in L(M) : a \in \sigma$ which is a contradiction with the assumption which requires $\exists \sigma_{-b} \cdot \sigma_a \in L(M)$.

Case 2: `exists(M,a)` returns **True**, `exists(M,b)` returns **True**, LCA is a sequence, b ancestors **do not** contain an exclusive branch and b appears before a in the tree order. The existence checks ensure via Statement 1.1 that $\exists \sigma_a \in L(M)$ and $\exists \sigma_b \in L(M)$. The ancestor check ensures that $\forall \sigma \in L(M) : b \in \sigma$ which implies that $\exists \sigma \in L(M) : a \in \sigma \wedge b \in \sigma$ as Def. 3.2 defines that only for exclusive choice branches can a leaf node not appear in a trace of a language. Furthermore, as the LCA is a sequence and b appears before a in the tree order, it holds that $\forall \sigma \in L(M) : a \notin \sigma \vee \sigma(b) < \sigma(a)$ which contradicts with the assumption.

Case 3: `exists(M,a)` return **True**, `exists(M,b)` returns **True**, LCA is a sequence, b ancestors **do** contain an exclusive, the MCA is a sequence, and b appears before a in the tree order. As in Case 2, the existence checks ensure existence in some traces. Unlike in Case 2, b can be on an exclusive branch, so it does not have to exist in every trace. However as the MCA is a sequence and the b appears before a in the tree order, Def. 3.2 ensures that $\forall \sigma \in L(M) : a \in \sigma \implies \sigma(b) < \sigma(a)$ which contradicts the assumption and is the original statement.

Statement 3: Assume M , a , b , t such that `max_time_between(M,a,b,t)` returns true, while $\exists \sigma_a \in L(M) : b(t) - a(t) > t$. The CP returns true iff: `exists(M,a)` returns true, `exists(M,b)` returns true, $\exists T \in \text{timeouts}$ `exists(M): cancel_last(M, T, b)`, and $T_t = t$. By Statement 1.1, the existence CP ensure that $\exists \sigma_a \in L(M)$ and $\exists \sigma_b \in L(M)$. Further, a timeout T

with send data value T_t equaling t is identified in a cancel last relationship (see Fig. 6) with b . As the `cancel_last` pattern looks for a process tree fragment matching to $\wedge_l(\rightarrow_p(T), \rightarrow_p(M', b, M'))$ where M' are arbitrary process trees. According to Def. 3.2 this ensures that $\forall \sigma \in L(M) : T \in \sigma \vee b \in \sigma$. Combining this with the assumption we get that $\exists \sigma_a \in L(M) : b(t) - a(t) > s$. However, T is in parallel to b and the only activity in its branch, while b can be at any point in its branch. So the start event of T is lesser or equal to b and by assumption its end time is at least t afterwards. However in that t the timeout would timeout and cancel b so it could not appear in σ_b which contradicts with the assumption.

Statement 4: Given the assumption that B is a correctly identified branch by the `condition_finder`, assume M, a, c such that `condition_eventually_follows(M,c,a)` returns true, while $\exists \sigma \in L(M), \exists i \in \mathbb{N}_0, \forall l \in \mathbb{N}_0 : \text{sat}(\sigma[i], c) \wedge \sigma[i+l] \neq a$. `Condition_eventually_follows(M,c,a)` returns true, iff: `exists(B,a)` returns true and, by extracting all data objects d in c to identify at which activities α in $A \subseteq \Sigma$, condition c can be satisfied and returning true iff all α_d are prior to branch B which ensures that $\forall d \in c, \forall \alpha \in A : \alpha_d \implies \sigma(\alpha) < \sigma(B)$. If there are no α_d , the Start Activity is assumed to receive d for which $\sigma(\text{Start}) < \sigma(B)$ is trivially satisfied. As we check for all α_d we know that $\sigma[i]$ has to be in A so $\sigma(\sigma[i]) < \sigma(B)$. Furthermore, as $a \in L(B)$ by Statement 1.1., it holds that $\sigma([\sigma[i]]) < \sigma(a)$, so it has to hold that $\exists i, l \in \mathbb{N}_0 : \sigma[i+l] = a$ which contradicts the assumption. $\square$

5. Evaluation

We provide a prototypical implementation of PTV in Sect. 5.1, apply it to a data set in Sect. 5.2, consider a comparative evaluation in Sect. 5.3, and present the results of a user study regarding its usability in Sect. 5.4 .

5.1. Implementation

AST-based requirements are verified on process trees in the Cloud Process Execution Engine (CPEE⁹). The CPEE supports an extensive range of BPMN element types, utilizes a process tree structure called CPEE-trees for representing executable process models, and enables live verification through

⁹<https://cpee.org/>, accessed 2025-03-05

a subscriber functionality that publishes the updated process tree to a REST-based subscriber whenever the process model is modified. The general structure of the PTV can be seen in Fig. 7 with complete documentation available on GitHub¹⁰.

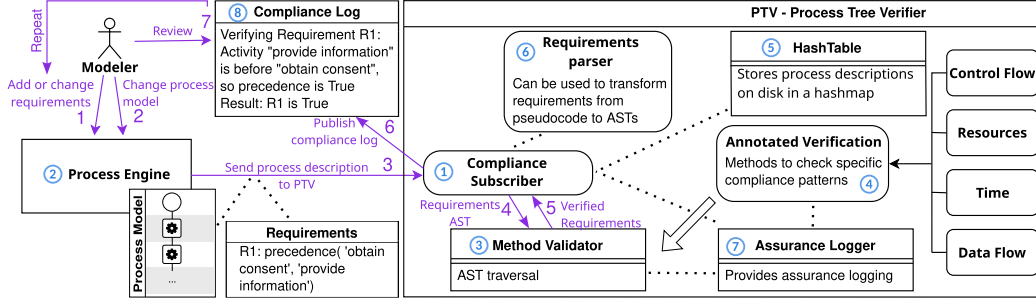


Figure 7: General structure of the PTV

The main module of the PTV is the compliance subscriber ①, which defines an endpoint towards which the process engine ② publishes the updated process descriptions containing the process tree, the encoded requirements and additional metadata (instance ID, author, ...) whenever the process modeler changes the process or updates the requirements. The requirement ASTs are traversed by the Method Validator class ③, which inherits the compiler's tree traversal methods to evaluate any ASTs where $\theta \subseteq CP$ defined in the annotated verification module ④. To improve code clarity and increase extendability, not all CP are part of the annotated verification module. Instead, lower-level abstraction checks (such as the compare ACP from Sect. 3) are organized into different utility modules for control flow, resources, time, and data flow. The PTV also contains a Hash Table class ⑤ for storing previous versions of process descriptions on disk and a requirements parser module ⑥ to parse simpler pseudo-code requirements into an AST string.

As motivated in the introduction, an additional advantage of structural, AST-based verification is the opportunity to trace all verification steps to highlight which requirements are still violable at design time and support the process modeler with identifying violations and restoring compliance. This functionality is achieved through the Assurance Logger class ⑦, which inherits from the standard Python logger class. It documents the verification

¹⁰<https://github.com/JohannesLbck/ProcessTreeVerify>, accessed 2025-03-05

steps, tracks missing activities, and calculates an assurance level by reducing the assurance whenever a CP is verified, which remains violable after verification. For example, the timed alternative CP presented in Sect. 3 reduces the assurance whenever the timeout endpoint identified in a cancel first relationship with the other activities is passed a data object instead of a digit since this means that the value of the timeout cannot be predicted at design time, i.e., the timed alternative is still violable even if the alternative activity is found on the alternative branch. A process modeler can acknowledge this warning and either fix a potential syntax error in the timeout or ensure that the value of the data object passed to the timeout will be correct at run time. The compliance logger automatically generates both simple compliance log for direct feedback to modelers (see ⑧ or the PTVLogs repository¹¹) and complex logs in a custom XES format compatible with standard miners (available online¹²). The custom XES format contains multiple instance and activity notions, to enable mining the process of verifying a single requirement, the process of verifying a single process, and the process of modeling a compliant process while supported by the PTV tool.

5.2. Application to Custom Data Set

Real-world datasets that are used for evaluating process mining and ex-post compliance verification techniques usually consist of event logs instead of (executable) process models, and associated requirements are rare [30]. Thus, we create a custom composite dataset¹³ of 11 processes with 46 requirements out of example processes/requirements used in related work. For this we identified contributions by searching google scholar and lens.org using the keywords *Business Process AND Compliance* and analyzed 20 contributions to extract processes with associated requirements.

Processes are included in the composite dataset if at least one of the requirements stated in the paper can be verified through the process. That means the requirements had to be relevant to the model shown in the paper and not use vague terms (“The information provided to the donor should be understandable” cannot be verified on the process model). The 46 requirements cover the typical process perspectives of control flow, resources, time,

¹¹<https://power.bpm.cit.tum.de/PTVLogs/>, accessed 2025-08-13

¹²<https://cpee.org/comp-log/>, accessed 2026-02-24

¹³<https://github.com/JohannesLbck/ProcessTreeVerify>, Composite Dataset directory contains data and additional documentation

Origin	Exec. Time	# Req.	Verifiable?	Fixable?
[4]	1ms	6	Yes	Yes
[15]	3ms	7	Yes	Yes
[31]	1ms	3	Yes	Yes
[5]	1ms	5	Yes	Yes
[32]	68 / 2ms	2	Yes	Yes
[16]	1ms	3	Yes	Yes
[33]	1ms	4	Yes	Yes
[34]	1ms	2	Yes	Yes
[35]	1ms	4	Yes	Yes
[36]	1ms	3	Yes	Yes
[37]	2ms	7	Yes	Yes

Table 4: DataSet evaluation results

and data flow. For each process, we translate the model into an executable process tree and the requirements into ASTs. To keep the process trees as close to the original models as possible, we keep activity and data object labels as well as the control flow identical, while adding data flow by considering which activities need to read/write which data objects to enable data condition dependent branches. By checking the associated requirements on all processes in the composite dataset and using violations identified by the PTV to adjust the processes until they are compliant, we verify that (a) all the process-related requirements used in related work are verifiable with process tree verification and (b) the verification is reliable and consistent, i.e., the results are deterministic, each check yields the same result for the same input in nearly the same execution time. Table 4 contains an overview of the evaluation results using the composite dataset, including execution times, number of requirements, and whether all the requirements were verifiable and compliance could be restored by checking the generated compliance log.

Execution times were logged using the PTV deployed on consumer-grade hardware (Nvidia RTX 4090, AMD Ryzen 9 7950X). As seen in Tab. 4, the verification finishes in less than four milliseconds as long as the process does not contain any sub-processes (see [32] taking 68 ms). For the PTV to verify a process with sub-processes, the sub-process execution tree is automatically fetched from a URL and merged into the parent tree, introducing unpredictable network delays averaging 68 ms in testing. This can be avoided by checking the parent and sub-processes separately or by storing processes on disk. In general, while sufficient for live verification, the execution times should be regarded as upper bounds.

5.3. Comparative Evaluation

A comparative evaluation with other BPC approaches is challenging, as many lack accessible implementations [4, 33, 16, 17, 38], have inaccessible in-

interfaces [15], or are limited to predefined models [35]. Moreover, several use fundamentally different paradigms, such as ex-post analysis [39, 40], declarative process modeling [41, 6, 42], or mashup-based modeling [34], making direct comparison inappropriate. We identified [9] and [29, 13] as the only design-time approaches with a comparable and accessible implementation.

It is important to note that a comparative evaluation is still not fully appropriate due to several differences: [9] support fewer compliance patterns and verify petri nets, which requires manual transformation from process trees to petri nets for comparison. [29, 13] verify workflow nets with respect to goal models rather than pattern-based compliance verification of process trees, so constraints must be translated into goal models. Finally, execution time differences can arise from computational complexity, implementation differences, and confounding variables. To account for confounding variables, we executed each test 10 times and averaged the execution time. Accordingly, for the evaluation, we first transformed the running example process into a semantically equivalent, guarded petri net. For [29] we also created a version of the petri net without guards.

For the requirements, we split 8 requirements of the running example into 11 atomic constraints (R1.1, R1.2, ..., R6.2 in Tab. 5) and manually translated them into a custom xml specifications format for [9] and a goal model for [29]. The rules are selected to cover a range of compliance patterns and can be found in the ComparativeEval dictionary on GitHub. Not every rule could be translated as [9] do not consider time and resource constraints and [29] consider the control flow as given by the workflow net.

The verification results can be seen in Tab. 5, and the created artifacts are in the ComparativeEval dictionary on GitHub. $\times$ entries in Tab. 5 indicate that the requirement could not be encoded with the approach. Executing single-constraint PTV verification on the same machine as before returned sub-1ms results, so we instead ran all these tests on a less powerful machine (11th Gen Intel i7-1165G7 (8) CPU, Intel TigerLake-LP GT2 [Iris Xe G] GPU, 16GB DDR4 RAM). Finally, it is important to note that [29] identifies a smaller state space than [9] when passing the same petri net without guards (43 vs 48 states).

As expected, both model-checking approaches take significantly longer to verify the running example process, as their computational complexity is NP, compared to the polynomial-time complexity of the PTV approach. As an approach based on semantic annotations (cf., Sec. 6), [29] verifies properties much quicker than [9], as they do not consider the state space of data

Rule	PTV	[9]	[29]
R1.1:	2.1 $\pm$ 0.3 ms	72.7 $\pm$ 5.9 ms	$\times$
R1.2:	4.4 $\pm$ 0.7 ms	72.2 $\pm$ 4.0 ms	11.6 $\pm$ 2.27 ms
R2:	1.4 $\pm$ 0.5 ms	$\times$	9.6 $\pm$ 0.84 ms
R3.1	2.0 $\pm$ 0.5 ms	$\times$	$\times$
R3.2	2.6 $\pm$ 0.7 ms	77.7 $\pm$ 7.0 ms	$\times$
R4.1	4.0 $\pm$ 0.0 ms	$\times$	10.1 $\pm$ 1.20 ms
R4.2	1.0 $\pm$ 0.0 ms	$\times$	8.9 $\pm$ 0.74 ms
R5.1	5.7 $\pm$ 1.3 ms	75.9 $\pm$ 3.1 ms	$\times$
R5.2	3.5 $\pm$ 0.7 ms	71.2 $\pm$ 4.3 ms	$\times$
R6.1	1.4 $\pm$ 0.5 ms	$\times$	$\times$
R6.2	1.7 $\pm$ 0.5 ms	$\times$	$\times$
Σ	29.8 $\pm$ 4.4 ms	370 $\pm$ 14.6 ms	40.2 $\pm$ 5.05 ms
$\bar{R}$	2.7 $\pm$ 0.4 ms	73.9 $\pm$ 2.9 ms	10.05 ms
All	18.0 $\pm$ 2.1 ms	76.8 $\pm$ 6.3 ms	2387.2 $\pm$ 98.79 ms

Table 5: Comparing PTV with model checking approaches. Rules can be found in the ComparativeEval dictionary on GitHub. *All* refers to testing all applicable in one run. $\pm$ refers to the standard deviation.

objects and instead relies on the assumption that the semantic annotations (goal models) are correctly enforced at run time. Accordingly, [9] provides the strongest correctness guarantees through model checking, while considering data objects, but also has the lowest performance for single constraint verification. A surprising result is how quickly the state space of the goal model explodes, as seen in the 2387ms it takes to verify using [29] when four compliance rules are considered in a single goal model. While Sec. 4.1 somewhat predicts this result through the $r + r^2$ in the computational complexity analysis, it is likely that the manually created goal model contains errors which cause a larger goal model state space explosion than necessary.

Conclusively, PTV should be preferred when considering large/complex models, and semantic annotations are challenging to enforce at run time. If the process models are comparatively small/simple (no string data objects, no loops, etc) and semantic annotations are challenging to enforce at run time, then an efficient state exploration method like [9] should be preferred as they provide the strongest correctness guarantee. Finally, approaches based on semantic annotations, such as [29], should be preferred when it is assumed that semantic annotations can be easily enforced at run time. Finally, [29] emphasizes the need to verify vague requirements (e.g., *timely* execution or *understandable* data) that are not considered in our approach, as we assume operationalized requirements. The comparative evaluation results also validate the complexity analysis in Sec. 4.1, as patterns with higher computational complexity also had longer execution times.

5.4. User Study

The qualitative user study is based on a questionnaire with additional artifacts (available on GitHub) and evaluates whether the PTV can help process modelers identify compliance violations in processes and assist in restoring compliance. The 13 study participants were identified by sharing the questionnaire with other researchers and industry experts. They were randomly assigned to one of two groups and provided with two process models, i.e., donor registration and blood collection process, containing compliance violations, along with a list of requirements covering all process perspectives for each process. Depending on their group (7 participants in A, 6 in B), participants got access to a generated **compliance log** using the Assurance Logger of the PTV (cf. Fig. 7) for either the donor registration or blood collection process. As discussed, the compliance log directly identifies violations and indicates how compliance could be restored. The participants' role was to identify which requirements were violated in each process and describe how they would restore compliance for all violated requirements. They could select violated requirements using multiple-choice with an additional field for natural-language reasoning, while restoring actions could be suggested in natural-language fields. Identification and restoring results were evaluated regarding their correctness by considering an identification incorrect whenever (a) a participant identified an unviolated requirement as violated without giving a valid reason why it could be regarded as violated or (b) a participant did not identify a violated requirement. Meanwhile, restoration was considered incorrect whenever a participant identified a requirement as violated but could not provide a valid explanation for how compliance could be restored. After completing the task for each process, the participants answered additional survey questions regarding their experience in process modeling, the study execution, how helpful they found the PTV for identifying violations and restoring compliance, and additional features they would be interested in. Data was collected using Microsoft Forms and processed in CSV files, which are available on GitHub alongside the other study materials. The study was conducted in person with two prior participants (not included in the 13) to identify flaws in the study design, while the 13 participants carried out the tasks and completed the task/questionnaire without direct support from the authors. The in-person studies lasted 43 and 28 minutes, respectively, while the purely online studies lasted, on average, 83 minutes, indicating that participants took breaks during the studies. Participants did not receive any (monetary) incentives.

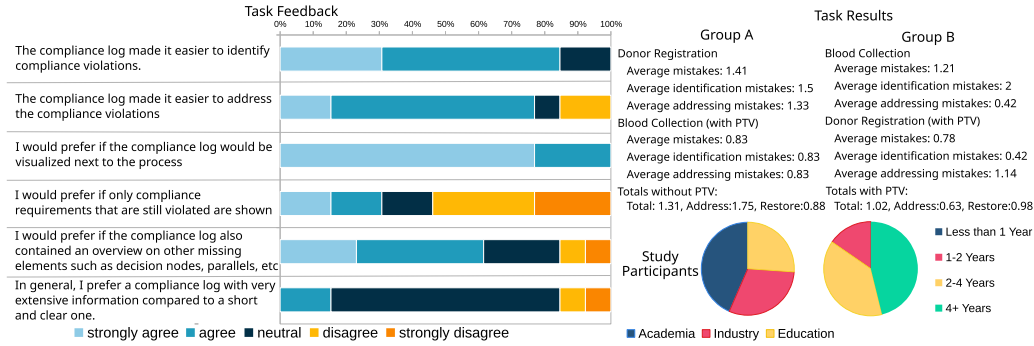


Figure 8: Overview user study results; Avg. mistakes rounded to the nearest 100th

The study results are promising while highlighting the need for better visualization (Fig. 8a). Of 13 participants, 11 strongly or weekly agreed that the PTV helped identify violations. The task results (Fig. 8b) also support this statement since, on average, the PTV reduced the amount of (a) incorrectly or (b) unidentified violations from 1.75 to 0.63 per process. Furthermore, 11 participants agreed that the PTV helped restore compliance. Interestingly, the results show that, on average, the PTV made participants slightly worse at restoring compliance (likely due to an over-reliance on the compliance log). For example, if an activity should directly follow a data condition, the compliance log indicated that the activity was missing, which meant several participants answered that the activity should be added without considering the data condition. However, in practice, the PTV would regenerate the log, informing users of the incorrect position. Regarding information density, participants somewhat preferred extensive compliance logs with additional information, and several highlighted the importance of transparency and interpretability. The main criticism was regarding the presentation: all participants agreed that the violations should be visualized next to the actual process instead of in the compliance log. Regarding the study execution, every participant provided a valid explanation of the study’s goal, while opinions regarding the complexity of the two processes varied: four found the donor registration more complex, three the blood collection process, and five considered them equally complicated. Most participants were from Academia (Fig. 8c) with 4+ years of experience in process modeling.

6. Related Work

Table 6 compares existing process compliance approaches with PTV, focusing on design time approaches. In line with compliance surveys such as [1, 37], there are no design-time tree-based compliance verification approaches. A process tree-based compliance approach by Awad et al. [24] focuses on resolving compliance violations. To this end, they identify which structures in a process tree can lead to violations of the `leads_to` CP and propose process tree modification algorithms to repair these violations. For the `leads_to` CP, we identify the same violating structures and use them to verify compliance. However, instead of considering repairing algorithms for violations of CP, we focus on verifying data, resource, and temporal CP. The survey in [3] identifies approaches most similar to PTV as execution-level abstraction approaches and highlights “an opportunity to integrate compliance-checking frameworks in BPMS to guarantee compliant executions” as explored by the COMPAS project, which resulted in the Compliance Request Language (CRL) [15]. CRL supports verification of BPEL models from the control flow, resources, temporal, and, with some extensions, data flow perspectives. The framework is not integrated into any process-aware information system [3], as BPEL is almost obsolete by now¹⁴. PROPOLS [38], an OWL-based approach used for BPEL model verification, lacks support for the resource and time perspectives.

Table 6: Overview of Compliance Frameworks analyzed regarding Expressivity

Approach	LC Phase	Formal	Type	Proofs	Control	A-DF	Process Perspectives		Time
							EL-DF	Resource	
BPMN-Q[4]	DT	CTL	I(MC)	yes	yes	yes	no	no	no
CRL[15]	DT	LTL/MTL	I(MC)	no	yes	no	no	yes	yes
DECLARE[42]	DT	LTL	D	yes	yes	yes	no	yes	yes
DCR[6]	DT	DL/HL	D	yes	yes	yes	no	yes	yes
eCRG[17]	DT/EP	FOL	I(LC)	no	yes	yes	no	yes	yes
PCL[16]	DT	MDL	I(MC)	yes	yes	yes	no	yes	yes
PNP[40]	EP	Petri-Net	I(LC)	no	yes	no	no	no	yes
PENELOPE[41]	DT	EC	D	yes	yes	yes	no	no	no
PROPOLS[38]	DT	OWL	I(MC)	no	yes	no	no	no	yes
PPSL[33]	DT	LTL	I(MC)	yes	yes	no	no	no	no
Rule Pattern[39]	EP	FOL/LTL	I(LC)	no	yes	yes	no	yes	yes
Status[43, 34]	DT	custom[43]	Mix	no	yes	yes	no	yes	yes
Gro. et al.[9]	DT	CTL	I(MC)	yes	yes	yes	no	no	no
Caball. et al. [29]	DT	LTS	I(MC)	yes	yes	no	no	no	no
Winter et al.[36]	DT	no	I(S)	no	yes	no	no	yes	no
Sun et al.[35]	DT	no	I(S)	no	yes	no	no	yes	no
PTV	DT	AST	I(S)	no	yes	yes	yes	yes	yes

LC: Life Cycle; DT: Design Time; EP: Ex-Post; I: Imperative; D: Declarative, LTS: Labeled Transition System, MC: Model Checking
A-DF: Annotated Data-flow; EL-DF: Execution Level Data-flow, LC: Log Checking, S: Structural

A central aspect of compliance frameworks is their verification strategy.

¹⁴<https://ode.apache.org/>, accessed 2024-10-30, was retired in 2019

As seen in Tab. 6, we differentiate between Model Checking (MC), Log Checking (LC), Declarative (D), and Structural (S) verification strategies.

Model checking is a design-time verification strategy where the state space of the process model is generated and explored to identify violating traces [4, 15, 16]. Due to parallel branches, loop operators, and (infinite) data object domains, compliance verification based on MC is an NP-complete problem as shown in [28]. In contrast, LC is in polynomial time complexity as it only requires checking a finite set of traces for violations, but can only be executed *ex post* [17, 40]. Accordingly, it can only give empirical predictions for subsequent process instance executions, so compliant execution is never guaranteed, and process modelers are not supported during modeling. In declarative modeling approaches [6, 42, 41], constraints are used to implicitly define the process execution, so compliance requirements can be used to directly define a compliant process. However, requirements extracted from legal documents often lead to under-specified processes [41, 42] which require additional constraints to enable execution. Adding the required additional constraints can again introduce constraint conflicts and compliance violations into the declarative model. Finally, structural approaches like PTV enable verification at design time in polynomial time, even when considering execution-level aspects such as data objects with infinite or unknown domains. However, proving the correctness of structural compliance verification results is challenging (cf. Sec. 4).

STATUS [34, 43] can be seen as a structural approach by presenting a design-time compliance approach that highlights (1) seamless integration with enterprise Information Systems (IS), (2) support for several formalisms and time notions, and (3) rule specification reuse by defining requirements as data-driven workflows called mashups in a structural approach. While PTV also achieves these requirements, it is less flexible compared to a mashup-based solution and requires more initial development effort as patterns have to be developed, but it provides stronger compliance and computational complexity assurances while requiring less manual work per process, as it does not rely on (semi-)manual creation of mashups per rule per process. Winter et al. [36] present a complete compliance management pipeline by extracting requirements from legal documents and mapping them to processes in a process repository. While their extraction achieves high accuracy for certain regulations (e.g., GDPR), they only support some control flow patterns and annotated resources. Sun et al. [35] improve the extraction method of [36] and present a user interface highlighting identified violations, but only

support the same control flow and annotated resource patterns. Both can be regarded as structural approaches as they rely on mapping terms from constraints to the terms used in the process models using similarity scores.

Groefsema et al. [9] present a MC approach that uses data condition abstractions to mitigate state space explosion. The approach based on petri nets and LTL encoding reduces the state space when considering conditioned requirements that are not globally enforced, but it does not consider time and resource aspects. They extend their work to further improve performance in [10] while still limiting the amount of loop executions, parallel/interleaving executions, and the domains of data objects (e.g., no support for strings as their domain is infinite) to limit the state space. PCL [16] uses Modal Defeasible Logic (MDL) to represent requirements described by the normative requirements typology of obligations, permissions, and compensations [19]. This allows for a more direct mapping between formalized requirements and legislation, but hinders understandability and automatic extraction due to its highly logical nature. Furthermore, they rely on semantic annotations [2]: predicates annotated to process elements to capture compliance properties without considering implementation details to enforce these properties at run time. Several approaches address the understandability problem through visual languages with a mapping towards a logic system [4, 17]. BPMN-Q [4] uses model checking to verify control flow and annotated data flow, similar to [17], who support control flow, time, data, and resource perspectives by verifying semantic annotations. In general, semantic annotations introduce an implementation gap between models and executed processes, but reduce the state space. Caballero et al. [29] present a MC approach for verifying compliance constraints defined in goal models. They transform both into Labeled Transition Systems (LTS) to create a composed goal and a process LTS to verify vague constraints (e.g., "appropriate measures", "timely confirmation"). As such, the goal models can be seen as a form of semantic annotation. To reduce state explosion, they consider only process models without complex branching, parallelism, data objects, or loops. PTV considers more complex processes while relying on annotations for resource requirements, but assumes operationalized requirements, where vague terms are defined (e.g., "timely" is replaced by a discrete duration or exact date).

DECLARE [42], DCR [6], and PENELOPE [41] are approaches for declarative process modeling that are well-suited for compliance due to their high flexibility [6, 42, 37]. Furthermore, DCR is supported through a proprietary process engine for the modeling, simulation, and execution of DCR

graphs [44]. However, as requirements extracted from constraints often under-specify processes, additional work is required to reliably extract compliant declarative processes from legal documents, and declarative processes are often regarded as more challenging to understand [44].

Finally, several approaches use process mining to conduct ex-post auditing (i.e., LC) and to verify compliance [39, 40], providing empirical predictions for subsequent process instance execution; i.e., compliant execution can never be guaranteed.

Analyzing both model-checking-based and structural design time compliance approaches, the state explosion problem seems to lead to trade-offs between flexibility and assurance levels (i.e., correctness) as well as between flexibility and automation. Highly flexible approaches [43] can be applied across all application domains and for all types of requirements, but require a high degree of manual effort and provide little guarantee of correct compliance verification.

Reducing flexibility leads to approaches like PTV, which require specific inputs (executable process trees with data objects and conditions) but can provide stronger compliance assurances and require less manual work. Model-checking approaches using semantic annotations [16] increase flexibility by representing complex implementation details using simple annotations. However, they provide lower assurances and require more manual work, as annotations have to be correctly mapped to implementation details at runtime. Finally, model-checking approaches that consider data objects [9, 10] can only be applied in very specific application scenarios (process models without loops, parallelism, or data objects with large/infinite domains) and cannot verify resource or temporal constraints. However, they provide the strongest correctness assurances through state exploration of both process and data states. For the illustration of these trade-offs in Fig. 9, the positions of the different approaches should be seen as a rough indicator and not an exact positioning.

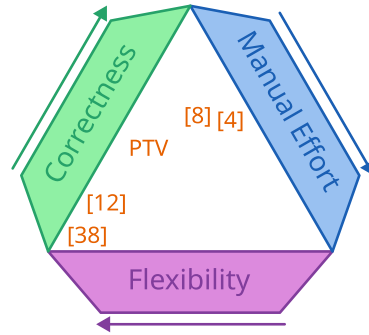


Figure 9: Design Time Compliance Tradeoffs

7. Discussion

We discuss limitations, threats to validity, generalizability, and correctness proofs.

Limitations: As a structural approach, PTV does not create the state space of a process model to produce formal behavioral correctness proofs. Formally proving the correctness of PTV is challenging, and correctness cannot be guaranteed for all patterns. As such, PTV reduces the risk of violating requirements and helps process modelers to enforce and restore compliance, but cannot guarantee full compliance by itself.

We show simplified proofs based on formal specifications for each CP and a formal process tree specification. However, these proofs also rely on the assumption that the implemented CPEE and the implemented PTV compliance pattern execute according to their specifications without side effects. Full proofs would additionally require verifying implementations in type-safe languages without side effects. Furthermore, verification of annotated resources is prototypical, and real-world adoption of PTV requires integration with a resource organization platform [27]. It is also important to highlight that the correctness proofs assume that endpoints behave according to their specifications, i.e., compliance violations do not originate from incorrectly implemented endpoints. These limitations hold for all design time compliance verification approaches.

In addition, PTV assumes availability of operationalized requirements where vague terms such as *timely* or *understandable material* are replaced with values/data objects such that ASTs can be extracted reliably using LLMs. In general, formalization and interpretation of legal texts, particularly defeasible norms with temporal validity, is a known challenge in research [19, 45], and our prototypical LLM-based extraction requires further evaluation.

Another limitation concerns the interpretability of the compliance requirements. Unlike elements of commonly used formalizations, such as LTL, not all compliance patterns, such as the `timed_alternative` pattern proposed in this paper, have widely agreed-upon meanings.

A conceptual limitation is the design-time focus. A complete BPC approach must also consider runtime monitoring and ex post analysis [8]. Finally, the evaluation dataset is synthetic and relatively small due to a lack of publicly available real-world datasets containing both process models and requirements, as also discussed in [10]. The comparative evaluation (cf. Sec. 5.3 with other BPC approaches was only possible for two approaches [9, 29],

relied on manual creation of several artifacts, and cannot account for confounding variables resulting from implementation details. A BPC challenge with standardized inputs and outputs as proposed in [3] could enable better comparative BPC analysis in the future

Threats to Validity: Regarding evaluation validity, the dataset was created by manually translating process models from related work into executable artifacts. Similarly, for the comparative evaluation, the executable process tree was manually translated into a guarded petri net. Despite independent work and discussion between authors, the resulting models may reflect biases in modeling style. An execution-time comparative evaluation is subject to many threats to validity, such as cache/memory misses, other running programs, and implementation details. While we sought to account for several by running each test 10 times and averaging the results, we mainly rely on formal complexity analysis, validated by the experimental results, to highlight PTV’s performance.

In the user study, we sought to balance confounding variables by randomizing assignment. However, process complexity may have influenced results, even if participants did not agree on a more complex process. Finally, there is a potential bias in interpreting what constitutes a *violation* and what steps suffice to restore compliance. Accordingly, we regarded well-argued violations, such as data quality violations, as correctly identified. Regarding external validity, all 13 participants in the user study had substantial modeling experience, limiting the generalizability of the results. By splitting the participants into groups with the order of tasks reversed, we aimed to account for the lack of (monetary) incentives and participant fatigue, which could have impacted the user study results.

Generalizability: As an imperative verification approach PTV cannot verify declarative models. Considering imperative notations, PTV can be directly applied for control flow verification of block-structured workflow graphs, such as executable BPMN models created in Camunda (<https://camunda.com/>), through a transformation to process trees [11]. The data perspective is more challenging, since BPMN only defines that a activity writes/reads a data object without considering execution-level abstraction details. Accordingly, data condition CPs can be verified, while data reads/write CPs depend on the exact notation style. Similarly, for resource and time annotations there is no unified execution-level notation, so the implemented CPs are CPEE-specific. Conceptually, all presented algorithms can be applied to any process tree based representation.

8. Conclusion

The Process Tree Verification (PTV) approach is based on structural compliance verification through intertwined traversal of process models represented as process structure trees and compliance requirements represented as Abstract Syntax Trees. By design, PTV is a structural, deterministic, white-box compliance verification approach designed for use with semi-operationalized requirements and both conceptual and executable process models. Since the verification has polynomial-time complexity, PTV supports both ex-ante and live verification, making it suitable for environments where process models or requirements frequently change. The use of AST encoding using the presented compliance patterns allows for performant, trustworthy, and traceable compliance verification of control flow, time, annotated resources, and (execution-level) data flow requirements through the creation of a compliance log that explains verification steps to process modelers, while highlighting why requirements are not yet enforced at design time (i.e., violable requirements [8]). Future work will address the discussed limitations and threats to validity through industrial collaborations, the development of PTV towards runtime compliance monitoring, more extensive user studies with different levels of experience and backgrounds, and discussions with legal experts.

AI Declaration. During the preparation of this work the author(s) used Grammarly to spelling check and Claude for creating documentation of code artifacts. After using these tools, the author(s) reviewed and edited the content as needed and take full responsibility for the content of the published article and associated artifacts.

Acknowledgments. This study was funded as part of the TRPro project by the German Research Association (Deutsche Forschungsgemeinschaft) as "Sachbeihilfe" under the project number 514769482.

References

- [1] M. Hashmi, G. Governatori, H. Lam, M. T. Wynn, Are we done with business process compliance: state of the art and challenges ahead, *Knowl. Inf. Syst.* 57 (1) (2018) 79–133. doi:10.1007/S10115-017-1142-1.
- [2] S. Sadiq, G. Governatori, K. Namiri, Modeling control objectives for

- business process compliance, in: *Business Process Management*, 2007, pp. 149–164.
- [3] H. A. López, T. T. Hildebrandt, Three decades of formal methods in business process compliance: A systematic literature review, *CoRR* abs/2410.10906 (2024). arXiv:2410.10906.
 - [4] A. Awad, M. Weidlich, M. Weske, Visually specifying compliance rules and explaining their violations for business processes, *J. Vis. Lang. Comput.* 22 (1) (2011) 30–55. doi:10.1016/J.JVLC.2010.11.002.
 - [5] S. Haarmann, A. Holfter, L. Pufahl, M. Weske, Formal Framework for Checking Compliance of Data-Driven Case Management, *J. Data Semant.* 10 (2021) 143–163.
 - [6] T. T. Hildebrandt, R. R. Mukkamala, Declarative event-based workflow as distributed dynamic condition response graphs, *arXiv preprint arXiv:1110.4161* (2011).
 - [7] M. Barrientos, K. Winter, S. Rinderle-Ma, Impact analysis of regulatory requirement changes on business process compliance, *Information and Software Technology* 194 (2026) 108079. doi:10.1016/j.infsof.2026.108079.
 - [8] L. T. Ly, S. Rinderle-Ma, K. Göser, P. Dadam, On enabling integrated process compliance with semantic constraints in process management systems - requirements, challenges, solutions, *Inf. Syst. Frontiers* (2012) 195–219doi:10.1007/S10796-009-9185-9.
 - [9] H. Groefsema, N. R. T. P. van Beest, A. Armas-Cervantes, Efficient conditional compliance checking of business process models, *Comput. Ind.* (2020). doi:10.1016/J.COMPIND.2019.103181.
 - [10] S. C. Tosatto, H. Burke, N. van Beest, H. Groefsema, A first and fast symbolic approach for data-aware business process compliance checking, *Information and Software Technology* (2026) 108047doi:https://doi.org/10.1016/j.infsof.2026.108047.
URL <https://www.sciencedirect.com/science/article/pii/S0950584926000364>

- [11] J. Vanhatalo, H. Völzer, J. Koehler, The refined process structure tree, in: *Business Process Management*, 2008, pp. 100–115.
- [12] J. Vanhatalo, H. Völzer, F. Leymann, Faster and more focused control-flow analysis for business process models through SESE decomposition, in: *Service-Oriented Computing - ICSOC*, Springer, 2007. doi:10.1007/978-3-540-74974-5_4.
- [13] J. Caballero-Villalobos, H. Baumeister, E. Paja, O. Kokoulina, H. A. López, Aligning processes with high-level requirements: Goal-model-based compliance checking (extended version), SSRN (Feb 2026). doi:http://dx.doi.org/10.2139/ssrn.6187219.
- [14] J. Loebbecke, J. Mangler, S. Rinderle-Ma, Tree-based compliance verification: Bridging the gap between compliance requirements and process execution, in: *Conceptual Modeling - 44th International Conference*, Vol. 16189, Springer, 2025, pp. 185–203. doi:10.1007/978-3-032-08623-5_10.
- [15] A. Elgammal, O. Türetken, CRL and the design-time compliance management framework, in: A. Polyvyanyy (Ed.), *Process Querying Methods*, Springer, 2022, pp. 285–311. doi:10.1007/978-3-030-92875-9_10. URL https://doi.org/10.1007/978-3-030-92875-9_10
- [16] G. Governatori, A. Rotolo, A conceptually rich model of business process compliance, in: *Asia-Pacific Conference on Conceptual Modelling*, 2010, pp. 3–12.
- [17] D. Knuplesch, M. Reichert, A visual language for modeling multiple perspectives of business process compliance rules, *Softw. Syst. Model.* 16 (3) (2017) 715–736.
- [18] J. Neuberger, L. Ackermann, H. van der Aa, S. Jablonski, A universal prompting strategy for extracting process model information from natural language text using large language models, in: *Conceptual Modeling*, 2024, pp. 38–55.
- [19] M. Hashmi, G. Governatori, M. T. Wynn, Normative requirements for regulatory compliance: An abstract formal framework, *Inf. Syst. Frontiers* 18 (3) (2016) 429–455. doi:10.1007/S10796-015-9558-1.

- [20] L. T. Ly, F. M. Maggi, M. Montali, S. Rinderle-Ma, W. M. P. van der Aalst, Compliance monitoring in business processes: Functionalities, application, and tool-support, *Inf. Syst.* 54 (2015) 209–234. doi:10.1016/J.IS.2015.02.007.
- [21] Q. Chen, K. Winter, S. Rinderle-Ma, Predicting unseen process behavior based on context information from compliance constraints, in: *Business Process Management - BPM*, Springer, 2023. doi:10.1007/978-3-031-41623-1_8.
- [22] S. J. J. Leemans, D. Fahland, W. M. P. van der Aalst, Discovering block-structured process models from event logs - A constructive approach, in: *Application and Theory of Petri Nets and Concurrency*, Vol. 7927 of *Lecture Notes in Computer Science*, Springer, 2013, pp. 311–329. doi:10.1007/978-3-642-38697-8_17.
- [23] S. Rinderle-Ma, J. Mangler, D. Ritter, *Fundamentals of Information Systems Interoperability - Data, Services, and Processes*, Springer, 2024.
- [24] A. Awad, S. Smirnov, M. Weske, Resolution of compliance violation in business process models: A planning-based approach, in: *CoopIS*, Vol. 5870, Springer, 2009, pp. 6–23. doi:10.1007/978-3-642-05148-7_4.
- [25] J. Loebbecke, S. Rinderle-Ma, A framework for assessing overcompliance and undercompliance in business processes, in: *CoopIS*, Vol. 15535, Springer, 2025, pp. 202–220. doi:10.1007/978-3-032-15538-2_12.
- [26] F. M. Maggi, M. Montali, C. Di Ciccio, J. Mendling, Semantical vacuity detection in declarative process mining, in: *Business Process Management: 14th International Conference, BPM 2016, Rio de Janeiro, Brazil, September 18-22, 2016. Proceedings 14*, Springer, 2016, pp. 158–175.
- [27] S. Ihde, L. Pufahl, M. Völker, A. Goel, M. Weske, A framework for modeling and executing task-specific resource allocations in business processes, *Computing* (2022). doi:10.1007/S00607-022-01093-2.
- [28] S. C. Tosatto, G. Governatori, N. van Beest, Checking regulatory compliance: Will we live to see it?, in: *Business Process Management - BPM 2019*, Springer, 2019. doi:10.1007/978-3-030-26619-6_10.

- [29] J. Caballero-Villalobos, A. Burattin, H. A. López, High-level requirements-driven business process compliance, in: Business Process Management Forum, Springer, 2026, pp. 23–39.
- [30] A. Burattin, F. M. Maggi, A. Sperduti, Conformance checking based on multi-perspective declarative process models, *Expert Syst. Appl.* 65 (2016) 194–211.
- [31] I. A. Letia, A. Goron, Model checking as support for inspecting compliance to rules in flexible processes, *Journal of Visual Languages & Computing* 28 (2015) 100–121.
- [32] L. T. Ly, C. Indiono, J. Mangler, S. Rinderle-Ma, Data transformation and semantic log purging for process mining, in: *Advanced Inf. Syst. Eng.*, 2012.
- [33] A. Förster, G. Engels, T. Schattkowsky, R. V. D. Straeten, Verification of business process quality constraints based on visual process patterns, in: *Symposium on Theoretical Aspects of Software Engineering*, 2007, pp. 197–208.
- [34] Á. Bernal, F. Montero, C. Cabanillas, P. Fernandez, M. Resinas, STATUS: A low-code business process compliance management system, in: *BPM Demos*, 2024, pp. 141–145.
URL <https://ceur-ws.org/Vol-3758/paper-26.pdf>
- [35] X. Sun, S. Yang, C. Zhao, D. Yu, Design-time business process compliance assessment based on multi-granularity semantic information, *J. Supercomput.* 80 (4) (2024) 4943–4971. doi:10.1007/S11227-023-05626-0.
- [36] K. Winter, H. van der Aa, S. Rinderle-Ma, M. Weidlich, Assessing the compliance of business process models with regulatory documents, in: *Conceptual Modeling*, 2020, pp. 189–203.
- [37] A. Zasada, M. Hashmi, M. Fellmann, D. Knuplesch, Evaluation of Compliance Rule Languages for Modelling Regulatory Compliance Requirements, *Software* 2 (1) (2023) 71–120. doi:10.3390/software2010004.

- [38] J. Yu, T. P. Manh, J. Han, Y. Jin, Y. Han, J. Wang, Pattern based property specification and verification for service composition, in: *Web Information Systems Engineering*, 2006, pp. 156–168.
- [39] F. Caron, J. Vanthienen, B. Baesens, Comprehensive rule-based compliance checking and risk management with process mining, *Decis. Support Syst.* 54 (3) (2013) 1357–1369. doi:10.1016/J.DSS.2012.12.012.
- [40] E. R. Taghiabadi, D. Fahland, B. F. van Dongen, W. M. P. van der Aalst, Diagnostic information for compliance checking of temporal compliance requirements, in: *Advanced Information Systems Engineering*, 2013, pp. 304–320.
- [41] S. Goedertier, J. Vanthienen, Designing compliant business processes with obligations and permissions, in: *BPM Workshops*, 2006, pp. 5–14.
- [42] M. Pesic, H. Schonenberg, W. M. P. van der Aalst, DECLARE: full support for loosely-structured processes, in: *Enterprise Distributed Object Computing Conference*, 2007, pp. 287–300.
- [43] C. Cabanillas, M. Resinas, A. Ruiz-Cortés, A mashup-based framework for business process compliance checking, *IEEE Trans. Serv. Comput.* 15 (3) (2022) 1564–1577. doi:10.1109/TSC.2020.3001292.
- [44] S. Debois, T. T. Hildebrandt, M. Marquard, T. Slaats, The dcr graphs process portal., in: *BPM (Demos)*, 2016, pp. 7–11.
- [45] T. Novotna, T. Libal, An evaluation of methodologies for legal formalization, in: *International Workshop on Explainable, Transparent Autonomous Agents and Multi-Agent Systems*, 2022, pp. 189–203.